

Theory of Machine

~~Ergodic~~ Minds

Exploring Ergodicity, Symmetry, and Confidence in
Modern Neural Networks.

Alec Rogers
August 15, 2025

Table of Contents

Table of Contents	2
Abstract	3
Background	4
Methods	8
Results	10
Discussion	11
Conclusion	13
References	13
Appendix 1: Ergodic.py	14
Appendix 2: Model.py	40

Society has become highly dependent on minds, but we do not have a good theory of these minds.

~~This paper explores several different neural network paradigms that differ from prominent architectures in four main ways: sample normalization, weight ergodicity, network invertibility, and output certainty. **Sample Normalization** refers to the fact that the input consists of samples from a probability distribution, so the data should be treated as estimates of an input distribution rather than specific points. **Weight ergodicity** refers to the fact that weights are samples from a random (ergodic) process, rather than fixed constants. **Network invertibility** refers to the fact that the network architecture is bidirectional and ~~possibly~~ ^{partially} invertible. **Output certainty** refers to the fact that a network should know that it does not know, as opposed to knowing the probability of one answer as opposed to another. To illustrate these principles, the performance of several models implementing these paradigms is compared with a traditional model. All models are configured with a 20-neuron, single hidden layer and tested on the MNIST dataset.~~

neural architectures
which are more similar to
- humanistic
- natural.

Human minds and machine minds have considerable overlap in functionality, although their design goals are significantly different.

Several of the insights in this exploratory paper have to do with the notion of ergodicity (or random sampling) as described in measurement theory. From that point of view, the inputs and outputs of a neural network are samples from a probability distribution as opposed to numerical constants. This distribution can be characterized by a moment transformation of the input vector at a given time, which means that it can be completely described by a set of (central) moments. Although this is non-trivial to compute in its entirety, the first two moments (the mean and variance) can be computed easily.

As the training in this paper is based on reinforcement learning, the inputs and outputs are known, but the weights of the network are unknown. However, they are not unknown constants: they are samples from an unknown process. Therefore, the weights are also represented as a random distribution that is conditioned (or shaped) by the inputs and outputs, and about which we know only the mean and the variance.

Sample Normalization

To reiterate, the inputs, outputs, and hidden values of a network are samples of an ergodic process with a fixed mean and variance. This observation is used to transform (and thereby whiten) the data at every layer by removing the predicted mean and variance in an approximation of an *instantaneous moment transformation*. Unlike batch normalization, which removes the batch mean and batch variance of each element of an input vector x , sample normalization removes the mean and variance of the entire input vector at each time step. However, the sample mean and variance are appended to the input vector x (rather than being used to reshift and rescale the input after being multiplied by the weight matrix W , as in batch normalization). These two channels on the output of every neuron also allow some degree of visibility into the neural architecture, since collectively they represent an approximate topological map of neural activity.

Numerically, sample normalization performs the following operations on an input vector x :

$$x_m = \sum_i^n \frac{x_i}{n}$$

$$x_v = \sum_i^n \frac{(x_i - x_m)^2}{n}$$

$$X = \left[1, \frac{x - x_m}{x_v}, x_m, x_v \right]$$

Although this transformation yields a vector x that has two moments and a bias parameter in addition to the original weight matrix, it has the benefit of providing an approximately standard normal input vector (it is exactly standard normal except for the effect of the two (redundant) moments themselves). Ideally, the entire input vector x could be converted into its instantaneous moments using a moment transform, but doing so is numerically sensitive and computationally expensive. Note that averaging the instantaneous moment transform across time yields exactly the moments of the entire series (for a related concept pertaining to the autocorrelation of a series, see *Rogers 2003*).

Weight Ergodicity

Applying measurement theory to the network views the weights as samples from a random distribution. With that insight, the weight matrix W is represented as follows:

$$W = \mu M + \sigma V$$

where μ is the bias parameter and σ is the variance. The bias and variance parameterize a noisy signal with a mean of M and (gaussian) measurement noise V (thus V is unbiased and has unit variance). Note that the matrix M is analogous to the weight matrix in a typical neural network, and V is sampled from a standard normal distribution. For simplicity, the mean and variance of all weights are characterized here by a single *temperature* parameter for each layer that is updated via the ADAM algorithm. As with the learning rate, it may be desirable to have a temperature associated with each input weight, but doing so in this context would provide an unfair advantage over the normal network which is used as a measure of performance.

Another simplification in this context is to use the temperature parameter to determine both the bias and the variance. Given the temperature $temp$, the μ and σ parameters are calculated as follows:

$$\mu = 1 - temp$$

$$\sigma = temp$$

In this experiment, the temperature of the ergodic weight process begins at 1 and decreases to 0, using the ADAM algorithm to control the bias-variance tradeoff.

There are two significant advantages to using ergodic weights. The first advantage is that a learning rate is not necessary, since the biases have different contributions as a result of the randomness and converge as the temperature decreases. Thus, the adaptive learning rate schedule is reinterpreted as a process of simulated annealing. The second advantage is that the weights can be initialized to zero, since they move to different locations in weight space over time. It is hypothesized that this process of annealing will help the model to bounce out of local minima and saddle points within its weight space more often than taking a step in the direction opposite to the gradient, which is often prone to deterministic and oscillatory behavior.

As a further simplification provided by this framework, the standard dropout algorithm may be interpreted as variance in the bias parameter. Thus, even without explicitly setting neurons to zero with some frequency, the same type of regularization will occur naturally as a result of the difference in relative contribution of the weights.

Network Invertibility

The egocentric point of view tends to see perception as a passive process, while physics suggests that it is somewhat symmetric or at least bidirectional. That symmetry within a neural network can be approximated via the invertibility of its weight multiplications and activation functions. This bidirectionality provides an important role in autoencoders, an important network topology that performs a decoding step that is analogous to the reverse of the encoding step. In modern practice, the topological similarity of encoders and decoders in this context does not extend to shared weights. Here, we briefly examine the possibility of simultaneously training the network in both

the forward and reverse direction: the forward encoding step in which the outputs are predicted from the input, and the reverse decoding step in which the inputs are predicted from the (possibly predicted) outputs. For a related experiment in sharing the weight matrix between controller (policy) and critic (value) networks, see *Rogers et al 2001*.

In this particular experiment, the output is a one-hot encoded vector corresponding to the predicted MNIST digit, which is a clear example of a classification task that is not a bijective (invertible) function. To address this situation, the inputs are predicted from the outputs of the *penultimate* network layer; thus, the network is required only to have a bijective middle layer (and may have a surjective output layer). We characterize these components respectively as the *recognizer* and the *generalizer*. In this paper, we explore two forms of reversible networks: one which is two-way surjective and one which is bijective (where both the forward and backward computations share a single weight matrix).

Output Certainty

The error function in this study is modified form of cross-entropy loss. The reason for the modification is the observation that networks trained on cross-entropy loss emit a distribution that *must* classify the output. This does not allow the network to express the certainty that the input belongs to a given class or not; rather, it expresses the probability that the output belongs to one class *as opposed to another*. To remedy this situation, a variation of cross-entropy loss is used which blends standard cross-entropy loss with a product of cross-entropy loss and the norm of the prediction. As a result, an output value of zero will be penalized by cross-entropy loss, but any incorrect prediction will carry an additional penalty in proportion to its magnitude. The formula for the certainty-weighted cross-entropy version of the loss function is:

$$error = - \sum_{c=1}^N t_c \log(p_c)$$

$$certainty = |\hat{y}|$$

$$surprise = (\alpha) \cdot certainty \cdot error + (1 - \alpha) \cdot error$$

Where the parameter α adjusts the balance between penalizing ignorance and penalizing incorrect answers (the value α is chosen to be 0.5 in this experiment). The related formula for certainty-weighted MSE loss may be computed by substituting the error as follows:

$$error = (\hat{y} - y)^2$$

Methods

The software used to run these experiments is a combination of Python and PyTorch (the Python code is provided in the appendices).

The data is the MNIST dataset, chosen because it is simple and well-known. It consists of 60,000 training images and 10,000 test images, each 28x28 pixels in size, with a monochromatic bit depth of 256. Their target labels are provided as output. The data is preprocessed by removing its mean and variance, and is shuffled between trials.

For all paradigms, the input is a 28x28 element array and the output is a one-hot encoded representation of the target class. The architecture for all paradigms is a 20 hidden unit neural network. The batch size is 10, and each network is run for 9 epochs. To generate confidence intervals for model accuracy, each paradigm is run for 7 trials to generate error bars reflecting the standard deviation of the accuracy values.

The standard architecture that serves as a benchmark for the other paradigms uses a ReLU nonlinearity, and a learning rate of 0.01 that is updated via the ADAM method. Cross-entropy loss is used for the error function.

The base ergodic model uses weights initialized to zero and a single per-layer temperature parameter that governs the bias/variance trade-off of the ergodic weights. The temperature is updated with the ADAM method. Dropout is used for the middle layer, which means that both the bias and variance of the ergodic weights are set to zero with a frequency of 0.75.

Five models were evaluated: the base model and four models using the ergodic weight updating scheme. Of the models with ergodic weights, one model has simply ergodic weights, one has input normalization, one reversible model uses separate forward and backward layers, and one reversible model uses a single invertible weight matrix. For the invertible model, the inversion of the weight matrix is typically computationally expensive. Therefore, the weight matrix is represented in SVD form, as a rotation matrix followed by an eigenvalue matrix followed by another rotation matrix. In virtue of this sparsely-parameterized decomposition, the rotation matrices can be computed by Givens rotations, and the eigenvalue matrix stores only diagonal elements. Therefore, direct matrix inversion and its

computational cost are avoided. That said, doing so requires a more computationally expensive forward pass and a somewhat circuitous back propagation step.

The weight update equations for the ergodic model do not use a learning rate. So compared to traditional gradient descent equations that look as follows for a given learning rate parameter α :

$$\begin{aligned}err &= (y - \hat{y})^2 \\ \Delta_W &= \alpha(y - \hat{y}) \nabla_F x\end{aligned}$$

The ergodic weight updates are for the certainty-weighted MSE case are:

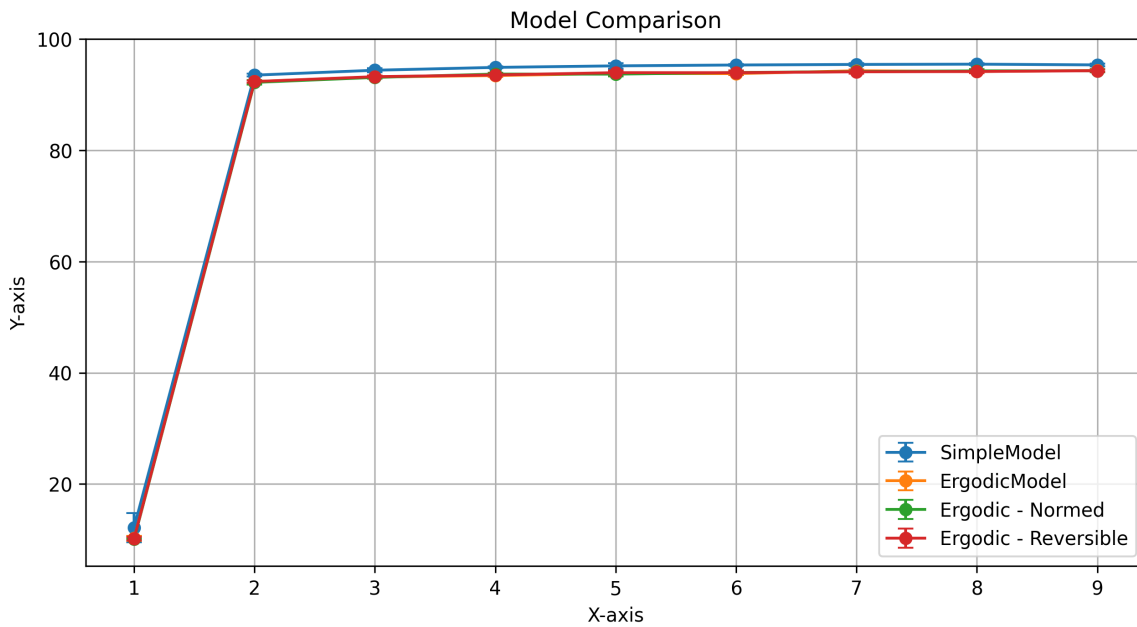
$$\begin{aligned}err &= (y - \hat{y})^2 \\ c &= \alpha \hat{y}^2 + (1 - \alpha) \\ e &= y - \hat{y} \\ \Delta_W &= (ce - \alpha \hat{y} e^2)x\end{aligned}$$

This equation demonstrates that the network is simultaneously trying to minimize error and “incorrect certainty”.

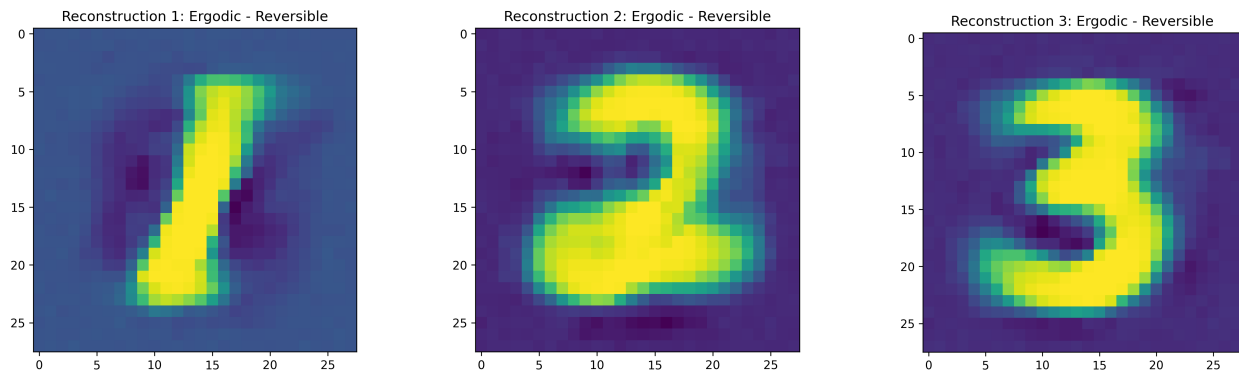
Results

Of the five paradigms originally proposed, only four are reported here, since the invertible network is not much better than chance when trying to simultaneously maximize predictions of the output and the input. It is likely the case that the weight update equations need to be carefully balanced, or some more intelligent cost function must be developed that simultaneously takes account of both forward and reverse predictions.

The most important result of this experiment is the model comparison with error bars, that demonstrates that the simple model performs slightly better than all proposed paradigms, and that all of the proposed paradigms perform almost identically:



Finally, although there is no invertible network to compare it to, we show the reconstruction of the input by running the network in reverse, which uses a separate weight matrix and MSE loss for input reconstruction:



Discussion

The benefits of the novel paradigms described in this paper are as follows:

- Random weight initialization is unnecessary, which makes performance less subject to arbitrary initialization. This is visually demonstrated by the error bars in the Model Comparison figure, which show that only the simple model (with random weight variation) has significant variation in performance.
- The learning rate is unnecessary, since its role is replaced with weight noise variance, just as the role of dropout is replaced with weight bias variance (which in this experiment is just the regular dropout schedule). Since dropout is folded into the weight adaptation schedule, an integrated algorithm that controls both weight bias and weight variance (and which uses separate parameters for each weight) is suggested for future experiments that follow this paradigm.
- Layer normalization (such as batch normalization) is replaced with sample normalization, which is conceptually more sound and does not suffer from the effects of batch size. It also whitens the data, which helps subsequent layers to focus on higher-order statistics.
- The error function encapsulates a measure of certainty in the output, as opposed to a measure of certainty between the output choices.

However, there were several experiments that my hardware was not fast enough to evaluate and a number of implementation issues that I did not have the time to address:

- One implementation issue was dropout, which set the temperature parameter to 0 in 75% of the trials. The problem with doing so is that the temperature parameter (denoted σ in the weight equations above) is applied across *all* weight means M in the hidden layer. This has no effect on ultimate performance, although it slows the speed of convergence by rendering 25% of the trials as no-ops.
- The Pearson correlation between the certainty and the accuracy for each digit is insignificant, which suggests that the error function did *not* induce the network to be less certain when it was less accurate. This probably points to a problem in the implementation of the error function: instead of using $certainty = |\hat{y}|$ to reduce the penalty for lack of certainty, we used $certainty = p_c$. This leads to prediction norms that are much higher than 1.0 (since softmax creates a unit-sum PDF at the output layer, while using the norm of the output vector to indicate certainty imposes a contradictory constraint). This might explain the lack of correlation between the certainty and accuracy, which theoretically should correlate quite strongly. Performing the experiment with corrected code (as by using a norm-

weighted MSE with only one zero) should yield better correlation between certainty and accuracy for each target class.

- Finally, the use of the autograd algorithm within PyTorch led to weights being tuned directly, which interferes with the simultaneous tuning of the temperature parameter in a way that is not explicitly captured in this paper. So while the addition of weight variance does make random weight initialization unnecessary, the intricacies of the interaction between the weight tuning and the temperature tuning is hidden behind the autograd mechanism.

Conclusion

Overall, I am optimistic about the strength of the philosophical grounding behind several of the paradigms introduced in this paper. However, due to several implementation issues, I do not feel that the hypotheses were well-tested. Further, even if they had been, the effectiveness of these neural network paradigms in both multilayer neural networks and with more challenging tasks is not clear.

References

Rogers, Shannon, Lendaris; 2001: *A comparison of DHP based antecedent parameter tuning strategies for fuzzy control*, Proceedings of the Joint 9th IFSA World Congress and 20th NAFIPS International Conference, IEEE

Rogers, A; 2003: *Analysis of the Instantaneous Estimate of Autocorrelation*, unpublished

Appendix 1: Ergodic.py

```
import math, os
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchviz import make_dot
from matplotlib import pyplot as plt
from datasets import load_dataset
from sklearn.decomposition import PCA
import pandas as pd
from vector_quantize_pytorch import VectorQuantize
from Model import message, SigmaLayer, ReversibleSigmaLayer
from Model import NormLayer, LinearLayer, AttentionLayer
from Model import GammaMem, CertaintyWeightedCrossEntropy, epsilon

class Data():
    train_input      = []
    train_output     = []
    validation_input  = []
    validation_output = []
    test_input        = []
    test_output       = []

    inputLength      = 128
    combinedTokens    = []

    def load(self, dataset):
        if dataset == "mnist":
            self.loadMNist()
        if dataset == "xor":
            self.loadXOR()
        if dataset == "tomatoes":
            self.loadTomatoes()
    def shuffle(self):
        rand_indx = torch.randperm(len(self.train_output))
        self.train_input = self.train_input[rand_indx][:]
        self.train_output = self.train_output[rand_indx][:]
    def loadMNist(self):
        df = pd.read_csv('../data/mnist_train.csv')
        train = df.values
        df = pd.read_csv('../data/mnist_test.csv')
        test = df.values
        self.train_input = torch.tensor(train[:, 1:]/255.0,
dtype=torch.float)
        mnistMean = torch.mean(self.train_input)
        #print("MNIST mean: ", mnistMean)
        self.train_input = self.train_input - mnistMean
        mnistSTD = torch.std(self.train_input)
        #print("MNIST std: ", mnistSTD)
        self.train_input = self.train_input / mnistSTD
```

```

        self.train_output = torch.zeros((train.shape[0],10),
dtype=torch.float)
        for i, ndx in enumerate(train[:, 0]):
            self.train_output[i][ndx:ndx+1] = 1.0
        self.test_input = torch.tensor(test[:, 1:]/255.0,
dtype=torch.float)
        self.test_input = (self.test_input -mnistMean)/ mnistSTD
        self.test_output = torch.zeros((test.shape[0],10),
dtype=torch.float)
        for i, ndx in enumerate(test[:, 0]):
            self.test_output[i][ndx:ndx+1] = 1.0
        #self.test_labels = self.test_labels.unsqueeze(0)
        self.validation_input = torch.tensor(test[:, 1:]/255.0,
dtype=torch.float)
        #self.validation_input = self.validation_input -
torch.mean(self.validation_input, dim=0)
        #self.validation_input = self.validation_input /
torch.var(self.validation_input, dim=0)
        self.validation_output = torch.zeros((test.shape[0],10),
dtype=torch.float)
        for i, ndx in enumerate(test[:, 0]):
            self.validation_output[i][ndx:ndx+1] = 1.0
        self.inputLength = 28 * 28
    def loadXOR(self):
        data = {
            "train": {
                "text": ["hello world", "hello there", "loving world",
"loving there" ], # nPercepts = 3
                "label": [[0], [1], [1], [0]]
                #"label": [[0, 1], [1, 0], [1, 0], [0, 1]]
            },
            "validation": {
                "text": ["hello world", "hello there", "loving world",
"loving there" ], # nPercepts = 3
                "label": [[0], [1], [1], [0]]
                #"label": [[0, 1], [1, 0], [1, 0], [0, 1]]
            },
            "test": {
                "text": ["hello world", "hello there", "loving world",
"loving there" ], # nPercepts = 3
                "label": [[0], [1], [1], [0]]
                #"label": [[0, 1], [1, 0], [1, 0], [0, 1]]
            }
        }
        self.train_input = data["train"]["text"]
        self.train_output = data["train"]["label"]
        self.validation_input = data["validation"]["text"]
        self.validation_output = data["validation"]["label"]
        self.test_input = data["test"]["text"]
        self.test_output = data["test"]["label"]
        self.processLM(data)
    def loadTomatoes(self):
        cache_file = "data/rottenTomatoes.data"

```

```

# Load or cache the pre-trained Word2Vec model
if os.path.exists(cache_file):
    print("Loading cached data...")
    data = torch.load(cache_file, weights_only=False)
else:
    print("Downloading data...")
    data = load_dataset("rotten_tomatoes")
    torch.save(data, cache_file)
self.processLM(data)
def processLM(self, data, permute=True):
    # tokenize the sentences
    train_tokens      = data["train"]["text"]
    train_labels      = data["train"]["label"]
    validation_tokens = data["validation"]["text"]
    validation_labels = data["validation"]["label"]
    test_tokens       = data["test"]["text"]
    test_labels       = data["test"]["label"]

    self.combinedTokens = train_tokens + validation_tokens +
test_tokens
    self.combinedTokens = list(set(self.combinedTokens))

    for i in range(len(self.train_input)):
        self.train_input[i] =
self.stringTensor(train_tokens[i])
        self.validation_input[i] =
self.stringTensor(validation_tokens[i])
        self.test_input[i] =
self.stringTensor(test_tokens[i])
        self.train_output[i] = torch.tensor(train_labels[i],
dtype=torch.float)
        self.validation_output[i] =
torch.tensor(validation_labels[i], dtype=torch.float)
        self.test_output[i] = torch.tensor(test_labels[i],
dtype=torch.float)

    if permute:
        self.shuffle()

def tokenize(self, TheLanguageModel):
    self.train_input =
TheLanguageModel.tokenize(self.train_input)
    self.validation_input =
TheLanguageModel.tokenize(self.validation_input)
    self.test_input =
TheLanguageModel.tokenize(self.test_input)
def data(self):
    data = {
        "train": {
            "text": self.train_input,
            "label": self.train_output
        },
        "validation": {
            "text": self.validation_input,

```



```

        "label": self.validation_output
    },
    "test": {
        "text": self.test_input,
        "label": self.test_output
    }
}

def getEmbeddingSize(self):
    return self.getInputSize(), self.getOutputSize()
def getInputSize(self):
    inShape = len(self.train_input)
    inputEmbeddingSize = self.train_input[0].shape[0]
    return inputEmbeddingSize
def getOutputSize(self):
    outShape = len(self.train_output)
    outputEmbeddingSize = self.train_output[0].shape[0]
    return outputEmbeddingSize
def stringTensor(self, string):
    ascii_values = [ord(char) for char in string]
    tensor = torch.tensor(ascii_values, dtype=torch.int8)
    #zero = torch.tensor(0, dtype=torch.int8)
    tensor = F.pad(tensor, (0, self.inputLength - tensor.size(0)),
'constant', 0) #ord(" "))
    assert tensor.shape[0]==self.inputLength
    return tensor
TheData = Data()

# A set of prototype vectors for each space
class VectorSet(nn.Module):
    nInput          = 0 # the number of inputs passed to the forward
method
    nVectors        = 0 # the number of outputs expected from the
forward method
    nDim            = 0 # the latent dimensionality
    vectors         = nn.Parameter(torch.randn([nVectors, nDim])) #
this is the actual matrix of vectors
    snapDistance    = 0.1 # vectors at or below this distance snap
to their corresponding prototypes
    eta             = 0.9
    codebookAct     = GammaMem(nVectors)
    frozen          = []
    returnOnlyFrozen = False
    freezingTemp    = 0.25
    # linear = nn.Linear(10, 5)
    # tracker = ColumnUsageTracker(linear, freezeThreshold=0.01)

    #def getSize(self):
    #    return self.nVectors

    def create(self, nInput, nVectors, nDim, customVQ=False,
signed=False):
        self.nInput    = nInput
        self.nVectors  = nVectors
        self.nDim      = nDim

```

```

        self.customVQ = customVQ
        self.signed = signed
    def updateWeights(self, embed_sum, cluster_size):
        # Zero out gradients for frozen indices
        weights = torch.ones(self.vq.codebook_size)
        if len(self.frozen) > 0:
            weights[self.frozen] = 0
            if not self.customVQ:
                self.vectors.grad[self.frozen,:] = 0
        return weights
    def freeze(self, activations = None):
        if activations is not None:
            # set the activations that are already frozen
            #unfrozenAct = [a if a not in self.frozen else 0 for a in
activations ]
            unfrozenAct = [activations[a].detach().numpy() if a not in
self.frozen else 0 for a in range(self.codebookSize)]
            #unfrozenAct = activations[unfrozen]
            #if len(unfrozen) > 0:
            indexMax = np.argmax(unfrozenAct)
            if unfrozenAct[indexMax] > self.freezingTemp:
                if not indexMax in self.frozen:
                    self.frozen.append(indexMax)
                    message(f"Frozen activation at {indexMax}")
    def addVectors(self, nVec=1, decay = 0.9):
        self.codebookSize = nVec
        self.codebookAct = GammaMem(nVec)
        self.frozen = [] #np.zeros(nVec)
        if self.customVQ:
            self.vq = VectorQuantize(
                dim = self.nDim,
                codebook_size = nVec,
                threshold_ema_dead_code = 1,
                #num_quantizers=1,          # Return the N nearest
quantized vectors
                decay = decay,              # the exponential moving
average decay, lower means the dictionary will change faster
                commitment_weight = 1.0,    # the weight on the
commitment loss
                #sample_codebook_temp=0.0, #
                #use_cosine_sim=True,
                #learnable_codebook=True,
                #ema_update=False,
                rotation_trick = True      # Set False to use the STE
gradient estimator or True to use the rotation trick.
            )
        else:
            # self.vq = VQLayer(
            #     nDim = nDim,
            #     codebookSize = nVectors,
            #     numQuantizers = nVectors)
            vec = torch.randn([nVec, self.nDim])
            self.vectors = vec[:, :]
    def forward(self, input):

```

```

# X should be of size batchSize x nInput x nDim
# Pad X if necessary
x      = input
batchSize = input.shape[0]
act     = torch.zeros([batchSize, self.codebookSize])
if self.customVQ:
    #if x.shape[-1] == self.nDim:
    #    x = torch.cat([x, torch.zeros([x.shape[0],
x.shape[1], TheObjectEncoding.objectSize])], dim=2)
    y     = torch.reshape(x, [-1, self.embeddingSize])

    quantized, indices, commit_loss = self.vq(y,
ema_update_weight=self.updateWeights)

    err = torch.norm(y-quantized, dim=1)
    err = torch.reshape(err, x.shape[0:2])
    quantized = torch.reshape(quantized, x.shape)
    # pick the nVector symbols with the smallest
reconstruction error
    # Get the top nVectors smallest reconstruction errors
    values_smallest, indices_smallest = torch.topk(err,
k=self.nVectors, dim=1, largest=False)
    for i in range(0, indices_smallest.shape[0]):
        for j in range(0, indices_smallest.shape[1]):
            if err[i,j] <= self.snapDistance:
                cosSim = self.unsignedAngle(x[i,
j, :].clone(), quantized[i, indices_smallest[i, j], :].clone())
                x[i, j, :] = quantized[i, indices_smallest[i,
j], :]
                # Is this filling the entire activation matrix
properly (on the LHS)?
                assert torch.all(indices_smallest <
self.codebookSize), "activation dimension is not correct."
                act[i, indices_smallest[i, j]] = cosSim # +
temp * random.random()
            else:
                #message("codebook miss")
                x[i, j, :] = quantized[i, indices_smallest[i,
j], :]
                #x[i, j, :] = torch.zeros( [1, 1,
self.embeddingSize])
        else:
            dists = self.codebookDistance(x)
            #x = torch.cat([x, torch.zeros([x.shape[0], x.shape[1],
TheObjectEncoding.objectSize])], dim=2)
            # Project the set of input vectors onto the basis vectors
(the vector set).
            # Then compute the column norm of the basis, which result
in activations (neuron power).
            # The top of those activations become the "Conscious" set
of the current space.
            for b in range(0, x.shape[0]):
                for v in range(0, x.shape[1]):

```

```

        nearestDist, nearestIdx = torch.topk(dists[b,v,:],
1, dim=-1, largest=True)
        err = nearestDist[0]
        if err <= self.snapDistance:
            #message("Using prototype vector")
            x[b,v,:] = self.vectors[nearestIdx,:]
            act[b, v] = nearestDist[v]
        #else:
        #    x[b,v,:] = x[b,v,:]
        # Update the codebook
        # Train the closest vector even if it is not used.
        if self.training:
            self.vectors[nearestIdx, :] = self.eta *
(self.vectors[nearestIdx, :] + (1-self.eta) * x[b,v,:])
            #self.vectors[nearestIdx, :] =
F.normalize(self.vectors[nearestIdx, :], p=2, dim=1)
            for b in range(0, x.shape[0]):
                self.codebookAct.delta(act[b,:])
                if self.returnOnlyFrozen:
                    unfrozen = [a for a in range(self.codebookSize) if a
not in self.frozen]
                    act[b, unfrozen] = 0
                    self.freeze(activations = self.codebookAct.get())
            return x

```

```

    # The following routine needs also to check if the inner product
    is positive,
    # otherwise the intersection of the hyperplanes outside of the
    unit circle
    # may indicate that the two regions are disjoint rather than parts
    of one another.

```

```

    # Therefore, there are three possible results:
    # part(a,b), part(b,a), or disjoint(a,b)
    def conceptParthood(A: torch.Tensor, B: torch.Tensor) -> float:
        # Normalize vectors A and B
        A_norm = A / A.norm()
        B_norm = B / B.norm()
        # Find orthogonal vector to both A and B (cross product for
3D, generalized for nD)
        cross_prod = torch.linalg.cross(A_norm, B_norm)
        orthogonal_vector = cross_prod / cross_prod.norm()
        # Calculate distance of intersection hyperplane from origin
        distance = orthogonal_vector.norm()
        # Normalize distance to get a measure between 0 and 1
        measure = torch.clamp(distance, 0, 1)
        return measure

```

```

    # Example usage
    A = torch.tensor([1.0, 2.0, 3.0])
    B = torch.tensor([3.0, 2.0, 1.0])
    measure = hyperplaneParthood(A, B)

```

```

# The following should be replaced with a mereological framework
# that operates on the voronoi cells of the LVQ as atoms.

```

```

def perceptParthood(A: torch.Tensor, B: torch.Tensor) -> float:
    """
    Computes the directional parthood ratio: how much of A is
    contained within B.

    Both A and B must be in  $[0, 1]^n$  and define axis-aligned
    hyperrectangles with origin.

    Parameters:
        A (torch.Tensor): vector defining hyperrectangle A (outer
corner)
        B (torch.Tensor): vector defining hyperrectangle B (outer
corner)
        eps (float): small value to avoid division by zero

    Returns:
        float: parthood ratio in  $[0, 1]$ 
    """
    A, B = A.clamp(0, 1), B.clamp(0, 1)
    ratio = torch.minimum(A / (B + epsilon), torch.ones_like(A))
    return torch.prod(ratio).item()

# Example usage
A = torch.tensor([1.0, 2.0, 3.0])
B = torch.tensor([3.0, 2.0, 1.0])
measure = hyperrectParthood(A, B)

def learn(self, x, target_idx, lr=0.01):
    """
    Simple LVQ prototype update
    x: (batchSize, nDim) input vectors
    target_idx: (batchSize,) indices of the "correct" prototype
    lr: learning rate
    """
    x = F.normalize(x, p=2, dim=-1)
    selected_vectors = self.vectors[target_idx] # (batchSize,
nDim )

    # LVQ update: move prototypes toward or away from inputs
    # We will implement "attraction" only for now (classic LVQ1)
    delta = lr * (x - selected_vectors)
    self.vectors.data[target_idx] += delta

    # Optional normalization to keep vectors on the sphere
    self.normalize()
# --- Vector Insertion ---
def replace(self, new_vectors):
    #assert(self.nVectors == self.vectors.shape[0])
    if self.customVQ:
        self.vq.codebook = torch.stack(new_vectors, dim=0)
    else:
        #vec = torch.randn([nVec, self.embeddingSize])
        #for i in range(0, nVec):

```

```

        #     vec[i, :] =
F.normalize(TheObjectEncoding(vec[i, :].unsqueeze(0).unsqueeze(0)),
p=2, dim=1)
        self.vectors = new_vectors
    def insert(self, new_vectors):
        """
        Insert one or more new vectors
        new_vectors: (nNew, nDim )
        """
        new_vectors = F.normalize(new_vectors, p=2, dim=-1)
        #for i in range(0, new_vectors.shape[0]):
        #    new_vectors[i, :] = TheObjectEncoding(new_vectors[i, :])
        self.vectors = nn.Parameter(torch.cat([self.vectors.data,
new_vectors], dim=0))
        self.nVectors = self.vectors.shape[0]
    # --- Vector Removal ---
    def remove(self, indices):
        """
        Remove vectors by index
        indices: list or tensor of indices to remove
        """
        mask = torch.ones(self.vectors.shape[0], dtype=torch.bool,
device=self.vectors.device)
        mask[indices] = False
        self.vectors = nn.Parameter(self.vectors.data[mask])
        self.nVectors = self.vectors.shape[0]
    # --- Fuzzy / Mereology Methods ---
    def norm(self, x):
        return torch.norm(x, dim=-1)
    def normalize(self, x=None):
        if x is None:
            with torch.no_grad():
                if self.signed:
                    self.vectors.data = F.normalize(self.vectors.data,
p=2, dim=-1)
                else:
                    self.vectors.data =
torch.maximum(torch.minimum(self.vectors.data, 1), 0)
                    self.vectors.data = F.normalize(self.vectors.data,
p=2, dim=-1)
            else:
                if self.signed:
                    x = F.normalize(x, p=2, dim=-1)
                else:
                    x = torch.maximum(torch.minimum(x, torch.tensor(1.0)),
torch.tensor(0.0))
                    x = F.normalize(x, p=2, dim=-1)
            return x
    def negate(self, x):
        return 1 - x
    def distance(self, x, y):
        N = self.codebookSize
        dist = (x.T @ y) / N
        return dist

```

```

    def codebookDistance(self, x):
        vec = self.vectors #[:, 0:-TheObjectEncoding.objectSize]
        # dist = self.angle(x.unsqueeze(2),
vec.unsqueeze(0).unsqueeze(0)) # (batchSize, nInput, nFeatures)
        dist = x @ vec.T / self.nDim
        return dist
    def unsignedAngle(self, x, y, dim=-1):
        #xShape = x.shape
        #yShape = y.shape
        #x = torch.reshape(x, (-1, xShape[-1]))
        #y = torch.reshape(y, (-1, yShape[-1]))
        cos_sim = F.cosine_similarity(x, y, dim=-1)
        #dot = torch.sum(x * y, dim=dim)
        #norm_x = torch.norm(x, p=2, dim=dim)
        #norm_y = torch.norm(y, p=2, dim=dim)
        #cos_sim = dot / (norm_x * norm_y + epsilon)
        # scale 0-1
        return 0.5 * (1-cos_sim) # scale [0-1]
    def equal(self, x, y):
        return 1.0 - self.angle(x, y)
    def part(self, x, y):
        return 1.0 - self.angle(x, y)
    def whole(self, x, y):
        return 1.0 - self.angle(y, x)
    def boundary(self, x, y):
        return torch.abs(self.part(x, y) - self.whole(x, y))
    def overlap(self, x, y):
        return torch.min(self.part(x, y), self.whole(x, y))
    def union(self, x, y):
        return torch.max(x, y)
    def intersection(self, x, y):
        return torch.min(x, y)
# A PassThroughVSet does no vector quantization (vectors come directly
from the input space)
class UnquantizedVSet(VectorSet):
    def create(self, nInput, nVectors, nDim):
        super().create(nInput, nVectors, nDim)
    def forward(self, x):
        y = x
        return y
    def reverse(self, y):
        x = y
        return x

class Space(nn.Module):
    name = ""
    vectorSet = []
    activation = None
    params = []
    layers = []

    def __init__(self, inputShape, nVectors, nDim, nOutput, vSet=None,
reversePass=False):
        super().__init__()

```

```

        self.inputShape    = inputShape
        self.outputShape   = [nVectors, nDim]
        self.nVectors      = nVectors
        self.nDim           = nDim # the latent dimensionality
        self.nOutput       = nOutput
        self.batchSize     = 0
        self.vectorSet     = []
        self.reversePass   = reversePass
        if vSet is not None:
            self.vectorSet.append(vSet)
        #self.createVectorSet()

    def lookup(self, x):
        activation = x[0]
        x = x.unsqueeze(0).unsqueeze(0)
        #x = torch.cat([torch.zeros([1,1,
TheObjectEncoding.conceptDim]), x[:, :, 1:]], dim=2)
        output, index, _ = self.vectors().vq(x)
        #output[:, :, 0:TheObjectEncoding.conceptDim] =
output[:, :, 0:TheObjectEncoding.conceptDim] * activation # multiply
the codebook vector by the activation
        return output

    def stats(self, x):
        #codebookUse = self.vectors().codebookUse
        #message(f"{self.name} Codebook activation:
{ np.sum(self.vectors().codebookAct.get()) }")
        return

    def vectors(self):
        # this is done to store by reference, since lists are mutable
entities (?)
        return self.vectorSet[0]

    def createVectorSet(self, quantized=False):
        if quantized:
            self.vectorSet.append(VectorSet())
            self.vectors().create(self.inputShape[0], self.nVectors,
self.nDim) # can be bigger than nVectors, cannot be smaller
            self.vectors().addVectors(nVec=self.nOutput)
        else:
            self.vectorSet.append(UnquantizedVSet()) # pass-through
            self.vectors().create(self.inputShape[0], self.nVectors,
self.nDim)

        # self.vectors().create(self.inputShape[0], self.nVectors,
self.nDim)
        # self.vectors().addVectors(nVec=LM.getSize(), LM=LM)

    def forwardBegin(self, x, reshape=False):
        self.batchSize = x.shape[0]
        if reshape:
            x = self.flatten(x, True)
            # assert list(x.shape) == [self.batchSize,
self.inputShape[0] *self.inputShape[1]]
        else:

```



```

        assert list(x.shape) == [self.batchSize,
self.inputShape[0], self.inputShape[1]]
        return x
    def forwardEnd(self, x, reshape=False):
        if reshape:
            x = self.reshape(x, True)
        else:
            # if x.shape[-1] == self.nDim: # zero-pad vectors without a
position.
            # x = TheObjectEncoding.forward(x)
            # x = torch.cat([x, torch.zeros([x.shape[0],
x.shape[1], TheObjectEncoding.objectSize])], dim=2)
            assert list(x.shape)==[self.batchSize,
self.outputShape[0], self.outputShape[1]]
            return x
    def reverseBegin(self, y, reshape=False):
        self.batchSize = y.shape[0]
        if reshape:
            y = self.flatten(y, False)
        else:
            assert list(y.shape) == [self.batchSize,
self.outputShape[0], self.outputShape[1]]
            return y
    def reverseEnd(self, y, reshape=False):
        if reshape:
            y = self.reshape(y, False)
        else:
            input, _ = self.getEmbeddedIO()
            assert list(y.shape) == [self.batchSize,
self.inputShape[0], input]
            return y
    def flatten(self, x, forward=True):
        if forward:
            x = x.reshape(self.batchSize, self.inputShape[0] *
self.inputShape[1])
        else:
            x = x.reshape(self.batchSize, self.outputShape[0] *
self.outputShape[1])
            return x
    def reshape(self, y, forward=True):
        if forward:
            y = y.reshape(self.batchSize, self.outputShape[0],
self.outputShape[1])
        else:
            y = y.reshape(self.batchSize, self.inputShape[0],
self.inputShape[1])
            return y
    def getParameters(self):
        return self.params
    def paramUpdate(self):
        for l in self.layers:
            l.paramUpdate()
class InputSpace(Space):
    name = "Inputs"

```

```

    def __init__(self, inputShape, nVectors, nDim, nOutput, vSet=None,
tokenizedInput=False):
        super().__init__(inputShape, nVectors, nDim, nOutput, vSet)
        # Size of the embedding is Batch Size (2) X Sequence Length
(3) X Embedding Dimension (100)
        self.input = torch.FloatTensor
        self.tokenizedInput = tokenizedInput
        #fullSize = outputShape[0]*outputShape[1]
        self.createVectorSet()
        # The world presenting itself
    def forward(self, input):
        self.batchSize = input.shape[0]
        assert list(input.shape) == [self.batchSize,
self.inputShape[0], self.inputShape[1]]
        self.input = self.vectors().forward(input)
        #self.input = self.lift.forward(self.input)
        assert list(self.input.shape) == [self.batchSize,
self.outputShape[0], self.outputShape[1]]
        return self.input
    def reverse(self, y):
        y = self.reverseBegin(y)
        #y = self.lift.reverse(y)
        #x = x.unsqueeze(0).unsqueeze(0)
        #x = torch.cat([torch.zeros([1, 1,
TheObjectEncoding.conceptDim]), x[:, :, 1:]], dim=2)
        #output[:, :, 0:TheObjectEncoding.conceptDim] = output[:, :,
0:TheObjectEncoding.conceptDim] * activation # multiply the codebook
vector by the activation
        self.input = self.vectors().reverse(y)
        #where, when = TheObjectEncoding.reverse(self.percepts)
        #self.input = self.reverseEnd(self.input, t)
        return self.input
class ConceptualSpace(Space):
    name = "Concepts"
    hasAttention = False
    hasNorm = False

    def __init__(self, inputShape, nVectors, nDim, nOutput, vSet=None,
reversePass=False, invertible=False, hasNorm=False):
        super().__init__(inputShape, nVectors, nDim, nOutput,
vSet=vSet, reversePass=reversePass)
        input, output = self.inputShape[0]*self.inputShape[1],
self.outputShape[0]*self.outputShape[1]
        self.invertible = invertible
        self.hasNorm = hasNorm
        if self.hasAttention:
            self.attention = AttentionLayer(self.outputShape[1],
self.outputShape[1])
        if hasNorm:
            self.norm = NormLayer(input, input+2)
            input += 2
        if reversePass:
            if self.invertible:

```

```

        self.sigma = ReversibleSigmaLayer(input, output,
permuteInput=False)
        self.forwardSigma, self.reverseSigma =
self.sigma.forward, self.sigma.reverse
        self.params = self.sigma.getParameters()
        self.layers += [self.sigma]
    else:
        self.sigma1 = SigmaLayer(input, output,
permuteInput=False)
        self.sigma2 = SigmaLayer(output, input,
permuteInput=False)
        self.forwardSigma, self.reverseSigma =
self.sigma1.forward, self.sigma2.forward
        self.params = self.sigma1.getParameters() +
self.sigma2.getParameters()
        self.layers += [self.sigma1, self.sigma2]
    else:
        self.sigma = SigmaLayer(input, output,
permuteInput=False)
        self.forwardSigma = self.sigma.forward
        self.params = self.sigma.getParameters()
        self.layers += [self.sigma]
    self.createVectorSet()
    def distance(self, x, y):
        # This is a dot-product distance that assumes the X are
normalized.
        # However, if the X are not normalized, the magnitudes may be
taken as a degree of certainty or knowing.
        # In which case, how do they grow from ignorance to certainty?
        # They would do so naturally if the input vectors are
normalized.
        # It would also be possible to use a tunable transfer
function.
        return x.T @ y
    def certainty(self, x):
        return x.T @ x
    # Knowing
    def forward(self, x):
        x = self.forwardBegin(x, reshape=True)
        if self.hasNorm:
            x = self.norm.forward(x)
        y = self.forwardSigma(x) # Pass through SigmaLayer
        if self.hasAttention:
            y = self.attention.forward(y)
        # Get the concept vectors from the codebook
        # replace some of the Dynamic Percepts with Static Percepts if
their distance is low
        y = self.vectors().forward(y) # This must be 4x8x24
        # Reshape the output tensor
        self.concepts = self.forwardEnd(y, reshape=True)
        return self.concepts
    # Visualizing
    def reverse(self, y):
        self.concepts = self.reverseBegin(y, reshape=True)

```

```

        if self.hasAttention:
            self.concepts = self.attention.reverse(self.concepts)
        self.concepts = self.reverseSigma(self.concepts)
        if self.hasNorm:
            self.concepts = self.norm.reverse(self.concepts)
        self.concepts = self.reverseEnd(self.concepts, reshape=True)
        return self.concepts
    @staticmethod
    def test():
        pass
class OutputSpace(Space):
    name = "Outputs"
    def __init__(self, inputShape, nVectors, nDim, nOutput, vSet=None,
reversePass=False):
        super(OutputSpace, self).__init__(inputShape, nVectors, nDim,
nOutput, vSet=vSet)
        #input, output =self.inputShape[1], self.outputShape[1]
        # the output is reshaped, so we can't use the above formula
        input = self.inputShape[0] * self.inputShape[1]
        output = self.outputShape[0] * self.outputShape[1]

        # output is 0 of reshaped and not embedded
        if reversePass:
            self.linear1 = LinearLayer(input, output)
            self.linear2 = LinearLayer(output, input)
            self.forwardLinear, self.reverseLinear =
self.linear1.forward, self.linear2.forward
            #self.linear = ReversibleLinearLayer(input, output)
            #self.forwardLinear, self.reverseLinear =
self.linear.forward, self.linear.reverse
            self.params = self.linear1.getParameters() +
self.linear2.getParameters()
            self.layers += [self.linear1, self.linear2]
        else:
            self.linear = LinearLayer(input, output)
            self.forwardLinear = self.linear.forward
            self.params = self.linear.getParameters()
            self.layers += [self.linear]
        self.createVectorSet()
    # Acting
    def forward(self, x):
        y = super().forwardBegin(x, reshape=True)
        # input is batchSize x nConcepts
        output = self.forwardLinear(y)
        output = self.forwardEnd(output, reshape=True)
        #self.output = self.vectors().output(self.percepts)
        return output
    # Being acted upon
    def reverse(self, y):
        y = self.reverseBegin(y, reshape=True)
        #assert list(y.shape) == [self.batchSize, self.outputShape[0],
self.outputShape[1]]
        y = self.reverseLinear(y)
        output = self.reverseEnd(y, reshape=True)

```

```

        return output

class BaseModel(nn.Module):
    name = "BaseModel"
    spaces = []
    reversePass = False
    vSet = None
    invertible = False
    hasNorm = False

    def create(self, nInput=32, nConcepts=20, nOutput=1):
        self.nInput = nInput
        self.nConcepts = nConcepts
        self.nOutput = nOutput
    def runTrials(self, numTrials=1, numEpochs=1, batchSize=10,
lr=0.001):
        acc = np.zeros([numTrials, numEpochs])
        print(f"\n\n==== {self.name} ====")
        for trial in range(numTrials):
            print(f"\nTrial [{trial + 1}/{numTrials}]")
            self.create(nInput=self.nInput, nConcepts=self.nConcepts,
nOutput=self.nOutput)
            acc[trial, :] = self.run(numEpochs=numEpochs,
batchSize=batchSize, lr=lr)
            np.savetxt(f"{self.name}.csv", np.array(acc), delimiter=",")
            return acc
        def run(self, numEpochs=1, batchSize=10, lr=0.001,
stoppingCriterion=0.1):
            """
            Runs the Transformer model.
            """
            trainLosses = [[],[]]
            validationLosses = [[],[]]
            minValidationLoss = math.inf
            testLosses = [[],[]]
            self.plot = True
            accuracy = []
            for epoch in range(numEpochs):
                print(f"Epoch [{epoch + 1}/{numEpochs}]")
                if epoch != 0: # Don't train on the first epoch
                    outErr, inErr, allOut, lastIn =
self.runEpoch(TheData.train_input, TheData.train_output, lr=lr,
batchSize=batchSize)
                    trainLosses[0].append(outErr)
                    trainLosses[1].append(inErr)
                    print(f"Train Loss: {outErr:.4f}, {inErr:.4f}")
                    #outErr, inErr, __ = self.runTest(TheData.validation_input,
TheData.validation_output)
                    #validationLosses[0].append(outErr)
                    #validationLosses[1].append(inErr)
                    #print(f"Validation Loss: {outErr:.4f}, {inErr:.4f}")

```

```

        outErr, inErr, allOut, lastIn =
self.runEpoch(TheData.test_input, TheData.test_output, lr=0,
batchSize=batchSize)
        testLosses[0].append(outErr)
        testLosses[1].append(inErr)
        #print(f"Test Loss: {outErr:.4f}, {inErr:.4f}")

        _, predicted = torch.max(allOut, 1)
        _, actual = torch.max(TheData.test_output, 1)
        total = predicted.size(0)
        correct = (predicted == actual).sum().item()
        accuracy += [correct / total]
        print(f"Test Accuracy: {100 * correct / total:.2f}%")

TheData.shuffle()

        if outErr > minValidationLoss + stoppingCriterion:
            print(f"Validation increasing")
            minValidationLoss = outErr
        if outErr < minValidationLoss:
            minValidationLoss = outErr

# Plot the loss over time
if self.plot:
    print(f"Final Stats:")
    self.plotLoss(trainLosses, validationLosses, testLosses)
    #self.plotNetwork()
    self.mnistReport()

    return accuracy
def paramUpdate(self):
    for s in self.spaces:
        s.paramUpdate()
def mnistReport(model):
    _, _, y_pred, last_x_pred = model.runEpoch(TheData.test_input,
TheData.test_output, lr=0)
    _, predicted = torch.max(y_pred, 1)
    _, actual = torch.max(TheData.test_output, 1)

    rCorrect = torch.zeros((10))
    for i in range(0,10):
        total = (actual == i).sum().item()
        correct = (actual==i) & (predicted==actual)
        nCorrect = correct.sum().item()
        rCorrect[i] = nCorrect / total
        print(f"Correctly predicted {i}: {rCorrect[i]}")

    plt.figure(figsize=(10, 5))
    plt.plot(range(0, 10), rCorrect, label="Error (per Input)",
marker='o')
    plt.xlabel("Digit")
    plt.ylabel("Accuracy")
    plt.title(f"Accuracy per Digit: {model.name}")
    plt.legend()

```

```

plt.grid(True)
# Save the plot as a high-resolution PNG (300 DPI)
filename = model.name + "Accuracy.png"
plt.savefig(filename, dpi=300, bbox_inches='tight')
plt.show()
def plotLoss(model, trainErr, valErr, testErr):
    """
    Plots the training, validation, and test losses over time.
    """
    plt.figure(figsize=(10, 5))

    plt.plot(range(1, len(trainErr[0]) + 1), trainErr[0],
label="Training Error", marker='o')
    #plt.plot(range(1, len(trainErr[1]) + 1), trainErr[1],
label="Training Error (Input)", marker='o')

    if testErr:
        plt.plot(range(1, len(testErr[0]) + 1), testErr[0],
label="Test Error", marker='x')
        #plt.plot(range(1, len(testErr[1]) + 1), testErr[1],
label="Test Error (Input)", marker='x')

    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(f"Error per Epoch: {model.name}")
    plt.legend()
    plt.grid(True)

    # Save the plot as a high-resolution PNG (300 DPI)
    filename = model.name + "Error.png"
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.show()
def plotActivations(model, figure=1, percepts=None, concepts=None,
symbols=None):

    # Plot training loss and some activation norms.
    fig = plt.figure(figure, figsize=(12, 4))
    fig.clf()

    if percepts is not None:
        p = percepts[-1, :, :].squeeze()
        pAct = torch.norm(p, dim=1).detach().numpy()
        plt.plot(pAct, marker='o', color='r')
        plt.xlabel("Activation")
        plt.ylabel("Percepts")
        plt.title("Perceptual Activation")

    if concepts is not None:
        c = concepts[-1, :, :].squeeze()
        cAct = torch.norm(c, dim=-1).detach().numpy()
        plt.plot(cAct, marker='o', color='b')
        plt.xlabel("Epoch")
        plt.ylabel("Concepts")
        plt.title("Conceptual Activation")

```

```

    if symbols is not None:
        s = symbols[-1, :, 0].squeeze()
        sAct = s.detach().numpy()
        plt.plot(sAct, marker='o', color='g')
        plt.xlabel("Epoch")
        plt.ylabel("Symbols")
        plt.title("Symbolic Activations")

    plt.tight_layout()
    plt.show()
def plotSpace(model):
    """
    Visualizes only the learned weight parameters.
    - For the perceptual layer, plots the 64 prototype weight
vectors (each 64-d)
    projected via PCA to 2D.
    - For the conceptual mapping (fc_p of the ConceptualLayer,
shape (16,64)),
    uses the same PCA transform and, for each of the 16 weight
vectors,
    draws a hyperplane in 2D. Each hyperplane is taken to be the
line orthogonal
    to the projected weight vector and passing through the mean
of the projected
    perceptual prototypes.
    - For the symbolic branch, computes the L2 norm of each of its
8 weight vectors (from fc_symbolic)
    and displays a lollipop plot.
    """
    # ----- Perceptual prototypes -----
    # prototypes: (64, 64)
    perc_weights = model.prototypes.data.cpu().numpy() # shape
(64,64)
    pca = PCA(n_components=2)
    perc_2d = pca.fit_transform(perc_weights) # (64,2)

    plt.figure(figsize=(18, 5))

    plt.subplot(1, 3, 1)
    plt.scatter(perc_2d[:, 0], perc_2d[:, 1], c='blue',
label="Perceptual Prototypes")
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("Perceptual Space Prototypes (64 points)")
    plt.legend()

    # Compute the mean of the projected prototypes.
    perc_mean = perc_2d.mean(axis=0)

    # ----- Conceptual hyperplanes -----
    # Use the fc_p weights from the ConceptualSpace (shape
(16,64))

```



```

        conc_weights = model.conceptual.fc_p.weight.data.cpu().numpy()
# (16,64)
        # Project these 16 weight vectors using the same PCA (note:
they are in the same space as prototypes).
        conc_proj = pca.transform(conc_weights) # (16,2)

        plt.subplot(1, 3, 2)
        plt.scatter(perc_2d[:, 0], perc_2d[:, 1], c='blue',
label="Perceptual Prototypes")
        # For each conceptual weight vector, draw its corresponding
hyperplane.
        x_vals = np.linspace(np.min(perc_2d[:, 0]) - 1,
np.max(perc_2d[:, 0]) + 1, 100)
        for i in range(conc_proj.shape[0]):
            n = conc_proj[i] # this is the projected weight vector
(treated as normal)
            # Define hyperplane:  $n \cdot (x - \text{perc\_mean}) = 0$ , i.e.
            #  $n_0(x - m_0) + n_1(y - m_1) = 0 \Rightarrow y = m_1 - (n_0/n_1)(x - m_0)$ , provided  $n_1 \neq 0$ .
            if np.abs(n[1]) > 1e-3:
                y_vals = perc_mean[1] - (n[0] / n[1]) * (x_vals -
perc_mean[0])
            plt.plot(x_vals, y_vals, '--', label=f"Hyperplane {i +
1}" if i == 0 else None, alpha=0.7)
            else:
                # Vertical line at  $x = \text{perc\_mean}[0]$ 
                plt.axvline(x=perc_mean[0], linestyle='--', alpha=0.7,
label=f"Hyperplane {i + 1}" if i == 0 else None)
        plt.xlabel("PC1")
        plt.ylabel("PC2")
        plt.title("Conceptual Hyperplanes (16 lines)")
        plt.legend()

        # ----- Symbolic branch: Lollipop plot -----
        # fc_symbolic: Linear(784,8) has weight of shape (8,784)
        symb_weights = model.fc_symbolic.weight.data.cpu().numpy() #
(8,784)
        symb_norms = np.linalg.norm(symb_weights, axis=1) # (8,)
        x_symb = np.arange(8)

        plt.subplot(1, 3, 3)
        plt.vlines(x_symb, 0, symb_norms, color='k', alpha=0.7)
        plt.scatter(x_symb, symb_norms, color='red', s=100, zorder=3)
        plt.xlabel("Symbolic Feature Index")
        plt.ylabel("L2 Norm")
        plt.title("Symbolic Weights (Lollipop Plot)")

        plt.tight_layout()
        plt.show()
    def plotNetwork(model):
        """
        Uses Torchviz to create a visualization of the network's
computation graph.
        The visualization is saved as 'BasicModel_graph.png'.

```

```

"""
#dummy_input = torch.randn(1, 28, 28)
_, _, output, input = model.runEpoch(TheData.test_input,
TheData.test_output, lr=0)
dot = make_dot(output, params=dict(model.named_parameters()))
dot.format = "png"
dot.render(f"graph_{model.name}")
print(f"Saved network graph as {model.name}_graph.png")
def plotErrorbars(model, acc):
    y = np.array(np.mean(acc, axis=1))
    y = np.expand_dims(y, axis=1)
    y_err = np.std(acc - y, axis=1)
    plt.errorbar(x, np.squeeze(y), yerr=y_err, fmt='-o',
label=model.name, capsize=4)
class SimpleModel(BaseModel):
    name = "SimpleModel"

    def create(self, nInput=32, nConcepts=20, nOutput=1):
        super().create(nInput, nConcepts, nOutput)
        self.hidden = nn.Linear(self.nInput, self.nConcepts) # First
layer
        self.relu = nn.ReLU() # Activation
        self.output = nn.Linear(self.nConcepts, self.nOutput) #
Output layer (linear)
    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x)
        x = self.output(x) # Linear output (no activation)
        return x
    def runEpoch(self, input, output, lr=0.01, batchSize=10):
        if lr:
            optimizer = torch.optim.Adam(self.parameters(), lr=lr)
            criterion = nn.CrossEntropyLoss() # MSELoss()

            allOutput = []
            allInput = []
            outErr = 0
            inErr = 0
            self.train(lr!=0)
            for i in range(0, len(input), batchSize):
                inputBatch = input[i:i + batchSize]
                outputBatch = output[i:i + batchSize]
                batchSize = len(inputBatch)

                # Combine the padded tensors
                inputTensor = inputBatch
                outputTensor = outputBatch

                # First run forward
                if lr:
                    optimizer.zero_grad()
                outputPred = self.forward(inputTensor)
                lossOut = criterion(outputPred, outputTensor)
                if lr:

```

```

        lossOut.backward()
        optimizer.step()
        outErr = lossOut.item()
        out = outputPred.clone().detach()
        if i==0:
            allOutput = out
        else:
            allOutput = torch.concat((allOutput, out), dim=0)
    return outErr, inErr, allOutput, allInput
class ErgodicModel(BaseModel):
    name = "ErgodicModel"

    def create(self, nInput=32, nConcepts=256, nOutput=32):
        super().create(nInput, nConcepts, nOutput)

        inputDim = 1
        conceptDim = 1
        outputDim = 1

        self.inputSpace = InputSpace([self.nInput, inputDim],
                                      self.nInput, inputDim,
                                      nOutput=self.nInput,
vSet=self.vSet)
        self.conceptualSpace = ConceptualSpace([self.nInput,
inputDim],
                                              self.nConcepts,
conceptDim,
                                              nOutput=self.nConcepts,

reversePass=self.reversePass,

invertible=self.invertible,
                                              hasNorm=self.hasNorm)
        # The input dimensionality of the output layer must be equal
to the sum of the output dimensionalities of the symbolic layers.
        self.outputSpace = OutputSpace([ self.nConcepts, conceptDim],
                                      self.nOutput, outputDim,
                                      nOutput = self.nOutput,
                                      reversePass = False)

        self.spaces += [self.inputSpace, self.conceptualSpace,
self.outputSpace]
    def forward(self, data):
        percepts = self.inputSpace(data)
        concepts = self.conceptualSpace(percepts)
        symbols = self.outputSpace(concepts)
        return symbols, concepts
    def reverse(self, concepts):
        percepts = self.conceptualSpace.reverse(concepts)
        data = self.inputSpace.reverse(percepts)
        return data, percepts
    def getOptimizer(self, lr=0.01):

```

```

        params      = self.inputSpace.getParameters() +
self.conceptualSpace.getParameters() +
self.outputSpace.getParameters()
        #params      = list(filter([], params))
        optimizer = optim.Adam(params, lr=lr) # For other params, if
any
        return optimizer
def runEpoch(self, input, output, lr=0.01, batchSize=10):
    if lr:
        optimizer = self.getOptimizer(lr=lr)
        #inputOptimizer = self.getOptimizer(lr=lr)

    criterionOutput = CertaintyWeightedCrossEntropy()
    #criterionOutput = CertaintyWeightedMSELoss()
    criterionInput  = nn.MSELoss()

    allOutput = []
    allInput  = []
    outErr    = 0
    inErr     = 0
    self.train(lr != 0)
    for i in range(0, len(input), batchSize):
        inputBatch  = input[i:i + batchSize]
        outputBatch = output[i:i + batchSize]
        batchSize   = len(inputBatch)

        # Combine the padded tensors
        inputTensor = inputBatch
        inputTensor = inputTensor.unsqueeze(2)
        outputTensor = outputBatch
        outputTensor = outputTensor.unsqueeze(2)

        # First run forward
        if lr:
            optimizer.zero_grad()
            outputPred, concepts = self.forward(inputTensor)
            lossOut = criterionOutput(outputPred.squeeze(),
outputTensor.squeeze())
            if lr:
                lossOut.backward()
                self.paramUpdate()
                optimizer.step()
            outErr = lossOut.item()
            outputPred = outputPred.clone().detach().squeeze()
            if i == 0:
                allOutput = outputPred
            else:
                allOutput = torch.concat((allOutput, outputPred),
dim=0)

        # Next run reverse
        if self.reversePass:
            if lr:
                optimizer.zero_grad()

```

```

        inputPred, percepts = self.reverse(concepts.detach())
        #lossIn = lossFnInput(eInput.squeeze().float(),
inputTensor.squeeze().float())
        lossIn = criterionInput(inputPred.squeeze(),
inputTensor.squeeze())
        if lr:
            lossIn.backward()
            self.paramUpdate()
            optimizer.step()
            inErr = lossIn.item()
            inputPred = inputPred.clone().detach().squeeze()
            # Can't save all input reconstructions
            allInput = inputPred
    return outErr, inErr, allOutput, allInput

def mnistReport(model):
    _, _, y_pred, last_x_pred = model.runEpoch(TheData.test_input,
TheData.test_output, lr=0)
    _, predicted = torch.max(y_pred, 1)
    _, actual = torch.max(TheData.test_output, 1)

    norms = torch.linalg.norm(model.outputSpace.linear.W, dim=0)
    rCorrect = torch.zeros_like(norms)
    for i in range(0,10):
        total = (actual == i).sum().item()
        correct = (actual==i) & (predicted==actual)
        nCorrect = correct.sum().item()
        rCorrect[i] = nCorrect / total
        print(f"Correctly predicted {i}: {rCorrect[i]}")
        print(f"Weight norm: {norms[i]}")

    # Compute the correlation matrix
    input_matrix = torch.stack((rCorrect, norms))
    correlation_matrix = torch.corrcoef(input_matrix)
    correlation_value = correlation_matrix[0, 1]
    print(f"Pearson Correlation: {correlation_value}")

    plt.figure(figsize=(10, 5))
    plt.plot(range(0, 10), rCorrect, label="Error (per Input)",
marker='o')
    plt.xlabel("Digit")
    plt.ylabel("Accuracy & Certainty")
    plt.title(f"Accuracy and Certainty: {model.name}")
    plt.legend()
    plt.grid(True)
    # Save the plot as a high-resolution PNG (300 DPI)
    filename = model.name + "Accuracy.png"
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.show()

    if model.reversePass:
        for i in range(0, 10):
            plt.figure(figsize=(10, 5))
            j = TheData.test_output[-i-1]

```

```

        _, num = torch.max(j, axis=0)
        plt.title(f"Reconstruction {num}: {model.name}")
        image = last_x_pred[9-i, :]
        image = np.reshape(image, (28, 28))
        plt.imshow(image)
        filename = model.name + f"_{num}_Reconstruction.png"
        plt.savefig(filename, dpi=300, bbox_inches='tight')
        plt.show()

def plotErrorbars(fn):
    x = list(range(1, 10))
    acc = np.loadtxt(f"{fn}.csv", delimiter=",")
    #acc = acc.T
    y = np.array(np.mean(acc, axis=1))
    y = np.expand_dims(y, axis=1)
    y_err = np.std(acc - y, axis=1)
    plt.errorbar(x, np.squeeze(y), yerr=y_err, fmt='-o', label=fn,
    capsizes=4)

def plotComparison():
    # Plot accuracies over all trials
    plt.figure(figsize=(10, 5))

    plotErrorbars("SimpleModel")
    plotErrorbars("ErgodicModel")
    plotErrorbars("Ergodic - Normed")
    plotErrorbars("Ergodic - Reversible")

    # Add labels and title
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")
    plt.title("Model Comparison")
    plt.legend()

    # Display the plot
    plt.grid(True)
    filename = "ModelComparison.png"
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.show()

if __name__ == "__main__":
    TheData.load('mnist')

    # Set the network's dimensionality
    nInput      = TheData.getInputSize()
    inputDim    = 1
    nConcepts   = 20
    conceptDim  = 1
    nOutput     = TheData.getOutputSize()
    outputDim   = 1

    numTrials   = 1
    numEpochs  = 1
    batchSize   = 10

```

```

Traditional = False
Ergodic     = True
Normed      = False
Reverse     = False
Invert      = False

if Traditional:
    TheSimpleModel = SimpleModel()
    TheSimpleModel.create(nInput=nInput, nConcepts=nConcepts,
nOutput=nOutput)
    TheSimpleModel.name = "SimpleModel"
    TheSimpleModel.runTrials(numTrials, numEpochs, batchSize)
if Ergodic:
    TheErgodicModel = ErgodicModel()
    TheErgodicModel.create(nInput=nInput, nConcepts=nConcepts,
nOutput=nOutput)
    TheErgodicModel.name = "ErgodicModel"
    TheErgodicModel.runTrials(numTrials, numEpochs, batchSize)
if Normed:
    TheErgodicModel = ErgodicModel()
    TheErgodicModel.create(nInput=nInput, nConcepts=nConcepts,
nOutput=nOutput)
    TheErgodicModel.hasNorm = True
    TheErgodicModel.name = "Ergodic - Normed"
    TheErgodicModel.runTrials(numTrials, numEpochs, batchSize)
if Reverse:
    TheErgodicModel = ErgodicModel()
    TheErgodicModel.create(nInput=nInput, nConcepts=nConcepts,
nOutput=nOutput)
    TheErgodicModel.reversePass=True
    TheErgodicModel.name = "Ergodic - Reversible"
    TheErgodicModel.runTrials(numTrials, numEpochs, batchSize)
if Invert:
    TheErgodicModel = ErgodicModel()
    TheErgodicModel.create(nInput=nInput, nConcepts=nConcepts,
nOutput=nOutput)
    TheErgodicModel.reversePass=True
    TheErgodicModel.invertible=True
    TheErgodicModel.name = "Ergodic - Invertible"
    TheErgodicModel.runTrials(numTrials, numEpochs, batchSize)

#plotComparison()

```

Appendix 2: Model.py

```
import warnings
import numpy as np
import matplotlib.pyplot as plt
import torch
import random
import torch.nn as nn
import torch.nn.functional as F
from vector_quantize_pytorch import ResidualVQ, VectorQuantize
from itertools import chain
import torch.optim as optim
import time

epsilon = 1e-7 # to avoid log(0)

class Message():
    def __call__(self, txt, newline="\n"):
        print(txt, end=newline)
message = Message()

#region Layers
class Layer(nn.Module):
    def __init__(self, nInput, nOutput, permuteInput=False):
        super(Layer, self).__init__()
        self.nInput = nInput
        self.nOutput = nOutput
        self.permuteInput = permuteInput
        self.batch = 0

    def freeze(self, learn=False):
        for param in self.parameters():
            param.requires_grad = not learn

    def permute(self, x):
        self.batch = x.shape[0]
        if self.permuteInput:
            x = torch.permute(x, (0, 2, 1))
        return x

    def unpermute(self, y):
        if self.permuteInput:
            y = torch.permute(y, (0,2,1))
        return y

    def getParameters(self):
        params = [p for n, p in self.named_parameters()]
        return params

    def paramUpdate(self):
        pass

    def forward(self, x, bias=None, temp=None):
        batch = x.shape[0]
        assert x.shape[1] == self.nSymbols
        #assert x.shape[2] == TheObjectEncoding.objectSize
        return x
```



```

    def reverse(self, y, bias=None, temp=None):
        batch = y.shape[0]
        assert y.shape[1] == self.nOutput
        return y
class ErgodicLayer(Layer):
    def __init__(self, nInput, nOutput, permuteInput=False):
        super().__init__(nInput, nOutput, permuteInput)
        self.bias = nn.Parameter(torch.tensor(0.0),
requires_grad=False)
        self.temperature = nn.Parameter(torch.tensor(1.0),
requires_grad=True)
        self.register_buffer('m', torch.tensor(0.0)) #
shape: []
        self.register_buffer('v', torch.tensor(0.0)) #
shape: []
        self.register_buffer('t', torch.tensor(0)) #
shape: []
        self.lr = 0.01
        self.beta1 = 0.9
        self.beta2 = 0.999
        self.epsilon = 1e-8
        self.dropoutRate = 1.0 # was erroneously set to 0.75

    def getParameters(self):
        params = [p for n, p in self.named_parameters() if n !=
"temperature"]
        #params = [p for n, p in self.named_parameters()]
        return params
    def setBias(self, bias=0.0):
        with torch.no_grad():
            self.bias.copy_(bias)
            self.temperature.copy_(1-bias)
    def setTemperature(self, temp=0.0):
        with torch.no_grad():
            self.bias.copy_(1-temp)
            self.temperature.copy_(temp)
    @torch.no_grad()
    def temperature_update(self):
        if self.temperature.grad is None:
            return # Nothing to do if no grad computed
        self.t += 1
        grad = self.temperature.grad.item()
        self.m = self.beta1 * self.m + (1 - self.beta1) * grad
        self.v = self.beta2 * self.v + (1 - self.beta2) * (grad ** 2)
        m_hat = self.m / (1 - self.beta1 ** self.t.item())
        v_hat = self.v / (1 - self.beta2 ** self.t.item())
        update = self.lr * m_hat / (v_hat.sqrt() + self.epsilon)
        self.temperature.data -= update
        self.temperature.grad.zero_()
        self.setBias(1.0 - self.temperature)
        if random.random() > self.dropoutRate:
            self.setBias(0.0)
            self.setTemperature(0.0)
    def paramUpdate(self):

```

```

        self.temperature_update()

class LinearLayer(Layer):
    def __init__(self, nInput, nOutput, hasBias=True, W=None):
        super(LinearLayer, self).__init__(nInput, nOutput)
        self.hasBias = hasBias
        if W == None:
            W = torch.eye(self.nInput, self.nOutput)
        self.W = nn.Parameter(W)
        self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
        self.bias = nn.Parameter(torch.zeros(1,nOutput))

    def forward(self, x, bias=1.0, temp=0.0):
        if temp:
            self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
            W = bias*self.W + temp*self.noise
            output = x @ W
            if self.hasBias:
                output += self.bias
            return output

    @staticmethod
    def test():
        nInput, nOutput = 3, 4
        W = torch.rand(nInput, nOutput)
        layer = LinearLayer(nInput=nInput, nOutput=nOutput, W = W)
        # note: x must be strictly positive for this to work
        input = torch.rand((1, nInput))
        output = layer(input)

        print(f"Input: {input}")
        print(f"After forward linear: {output}")

class ReversibleRotationLayer(Layer):
    def __init__(self, dim, naive=False, theta=None):
        super(ReversibleRotationLayer, self).__init__(dim, dim)
        self.dim = dim
        self.naive = naive
        if theta is None:
            theta = torch.randn(dim - 1) * 2 * torch.pi
        self.theta = nn.Parameter(theta)
        # For simplicity, we initialize noise deterministically.
        self.noise = nn.Parameter(torch.zeros(dim - 1))

    def givens_rotation(self, i, j, theta):
        # Build a rotation matrix of size (dim x dim) that rotates in
the (i,j) plane.
        R = torch.eye(self.dim, device=theta.device,
dtype=theta.dtype)
        c, s = torch.cos(theta), torch.sin(theta)
        R[i, i] = c
        R[j, j] = c
        R[i, j] = -s

```

```

        R[j, i] = s
        return R
    def rotation_matrix(self, bias=1.0, temp=0.0):
        R_total = torch.eye(self.dim, device=self.theta.device,
dtype=self.theta.dtype)
        for idx, angle in enumerate(self.theta):
            angle = bias*angle + temp * self.noise[idx]
            # Use consecutive indices. (Assumes len(theta)==dim-1.)
            R = self.givens_rotation(idx, idx + 1, angle)
            R_total = R @ R_total
        return R_total
    def forward(self, x, bias=1.0, temp=0.0):
        # For the non-naive implementation, apply each rotation
sequentially.
        if self.naive:
            # Naive: build full rotation matrix and multiply on the
right.
            R = self.rotation_matrix(bias, temp)
            return x @ R # x (batch, dim) multiplied by R (dim, dim)
        else:
            # Direct approach applying each Givens rotation
sequentially.
            x_rotated = x.clone()
            # Note: We assume rotations are applied in natural order.
            for idx, angle in enumerate(self.theta):
                angle = bias*angle + temp * self.noise[idx]
                c, s = torch.cos(angle), torch.sin(angle)
                xi = x_rotated[:, idx].clone()
                xj = x_rotated[:, idx + 1].clone()
                # Standard Givens rotation: [xi, xj] -> [c*xi - s*xj,
s*xi + c*xj]
                x_rotated[:, idx] = c * xi - s * xj
                x_rotated[:, idx + 1] = s * xi + c * xj
            return x_rotated
    def forwardTranspose(self, x, bias=1.0, temp=0.0):
        # For the non-naive implementation, apply each rotation
sequentially in transpose (inverse) form.
        if self.naive:
            # Naive: build full rotation matrix and multiply by its
transpose
            R = self.rotation_matrix(bias, temp)
            return x @ R.T # x (batch, dim) multiplied by R^T (dim,
dim)
        else:
            # Sequentially apply inverse Givens rotations
            x_rotated = x.clone()
            # Apply in reverse order for exact transpose behavior
            for idx in reversed(range(len(self.theta))):
                angle = bias * self.theta[idx] + temp *
self.noise[idx]
                c, s = torch.cos(angle), torch.sin(angle)
                # Inverse rotation: negate sine term
                xi = x_rotated[:, idx].clone()
                xj = x_rotated[:, idx + 1].clone()

```

```

        x_rotated[:, idx] = c * xi + s * xj
        x_rotated[:, idx + 1] = -s * xi + c * xj
    return x_rotated
def reverse(self, x, bias=1.0, temp=0.0):
    # Since rotations are orthonormal, the inverse is given by the
transpose.
    if self.naive:
        R = self.rotation_matrix(bias, temp).T
        return x @ R
    else:
        x_rotated = x.clone()
        # Reverse the sequence in the opposite order.
        for idx in reversed(range(len(self.theta))):
            angle = bias * self.theta[idx] + temp *
self.noise[idx]
            c, s = torch.cos(angle), torch.sin(angle)
            xi = x_rotated[:, idx].clone()
            xj = x_rotated[:, idx + 1].clone()
            # Inverse rotation: [xi, xj] -> [c*xi + s*xj, -s*xi +
c*xj]
            x_rotated[:, idx] = c * xi + s * xj
            x_rotated[:, idx + 1] = -s * xi + c * xj
        return x_rotated
def reverseTranspose(self, x, bias=1.0, temp=0.0):
    # This is the inverse of forwardTranspose()
    if self.naive:
        R = self.rotation_matrix(bias, temp)
        return x @ R # forwardTranspose naive used R.T, so its
inverse uses R
    else:
        x_rotated = x.clone()
        # Apply in forward order (opposite of reverse()) for
transpose-of-inverse
        for idx in range(len(self.theta)):
            angle = bias * self.theta[idx] + temp *
self.noise[idx]
            c, s = torch.cos(angle), torch.sin(angle)
            # Undo the negated sine from forwardTranspose
            xi = x_rotated[:, idx].clone()
            xj = x_rotated[:, idx + 1].clone()
            x_rotated[:, idx] = c * xi - s * xj
            x_rotated[:, idx + 1] = s * xi + c * xj
        return x_rotated
@staticmethod
def test():
    dim = 4
    theta = torch.rand(dim - 1) * torch.pi / 2
    nlayer = ReversibleRotationLayer(dim=dim, naive=True,
theta=theta)
    layer = ReversibleRotationLayer(dim=dim, naive=False,
theta=theta)
    x = torch.tensor([[1, 2, 3, 4]], dtype=torch.float32)

    # Test forward pass

```

```

b = 1
t = 0
nrotated_x = nlayer.forward(x, b, t)
rotated_x = layer.forward(x, b, t)
print(f"Layer Weights: {layer.rotation_matrix()}")
print(f"Original: {x}")
print(f"Rotated Naive: {nrotated_x}")
print(f"Rotated: {rotated_x}")
print(f"Naive - Non-naive difference: {torch.norm(nrotated_x -
rotated_x)}")

# Test reverse pass
ninverse_x = nlayer.reverse(nrotated_x, b, t)
inverse_x = layer.reverse(rotated_x, b, t)
print(f"Inverse Naive Rotation: {ninverse_x}")
print(f"Inverse Rotation: {inverse_x}")
print(f"Forward-Reverse check (naive): {torch.norm(x -
ninverse_x)}")
print(f"Forward-Reverse check (non-naive): {torch.norm(x -
inverse_x)}")
class ReversibleDiagonalLayer(Layer):
    def __init__(self, nInput, nOutput):
        super(ReversibleDiagonalLayer, self).__init__(nInput, nOutput)
        self.nInput = nInput
        self.nOutput = nOutput
        self.rank = min(nInput, nOutput)
        # Store the nonzero singular values (for the effective rank).
        self.lamda = nn.Parameter(torch.ones(self.rank))
        self.noise = nn.Parameter(torch.zeros(self.rank))

    def stabilize(self):
        self.lamda.data = torch.minimum(self.lamda,
torch.ones_like(self.lamda))
    def forward(self, x, bias=1.0, temp=0.0):
        # Compute the effective singular values.
        w = bias * self.lamda + temp * self.noise
        # Build the Sigma matrix S of shape (nOutput x nInput)
        S = torch.zeros(self.nOutput, self.nInput, device=x.device,
dtype=x.dtype)
        for i in range(self.rank):
            S[i, i] = w[i]
        # Multiply input x (shape (... , nInput)) on the right by ST,
so output is (... , nOutput)
        y = x @ S.T
        return y
    def reverse(self, y, bias=1.0, temp=0.0):
        w = bias * self.lamda + temp * self.noise
        # Inversion: We assume the effective mapping is invertible on
the first 'rank' coordinates.
        if self.nInput <= self.nOutput:
            x = y[..., :self.nInput] / w
        else:
            x_known = y / w

```

```

        pad_shape = list(x_known.shape[:-1]) + [self.nInput -
self.nOutput]
        x_pad = torch.zeros(pad_shape, device=y.device,
dtype=y.dtype)
        x = torch.cat([x_known, x_pad], dim=-1)
        return x
    @staticmethod
    def test():
        """
        Runs several tests on the ReversibleDiagonalLayer:
        1. Square case: nInput == nOutput.
        2. Wide-output case: nInput < nOutput.
        3. Tall-input case: nInput > nOutput.
        4. Batch test.
        For the tall-input case and batch test (when nInput >
nOutput), the tests force the extra input coordinates
        to zero so that the mapping is invertible.
        """
        print("Testing ReversibleDiagonalLayer...")

        # Test 1: Square case: nInput == nOutput.
        nInput, nOutput = 4, 4
        layer = ReversibleDiagonalLayer(nInput, nOutput)
        x = torch.rand((10, nInput))
        b, t = 1, 0
        y = layer.forward(x, b, t)
        x_rec = layer.reverse(y, b, t)
        assert torch.allclose(x, x_rec, atol=1e-5), (
            f"Square test failed for nInput={nInput},
nOutput={nOutput}\n"
            f"x: {x}\nx_rec: {x_rec}"
        )
        print("Square test passed.")

        # Test 2: Wide output: nInput < nOutput.
        nInput, nOutput = 3, 5
        layer = ReversibleDiagonalLayer(nInput, nOutput)
        x = torch.rand((10, nInput))
        # For wide output, all nInput coordinates are used.
        b, t = 1, 0
        y = layer.forward(x, b, t)
        x_rec = layer.reverse(y, b, t)
        assert torch.allclose(x, x_rec, atol=1e-5), (
            f"Wide output test failed for nInput={nInput},
nOutput={nOutput}\n"
            f"x: {x}\nx_rec: {x_rec}"
        )
        print("Wide output test passed.")

        # Test 3: Tall input: nInput > nOutput.
        nInput, nOutput = 5, 3
        layer = ReversibleDiagonalLayer(nInput, nOutput)
        x = torch.rand((10, nInput))

```

```

        # For the mapping to be invertible, force the extra dimensions
(nOutput: nInput) to zero.
        if nInput > nOutput:
            x[:, nOutput:] = 0.0
            y = layer.forward(x, b, t)
            x_rec = layer.reverse(y, b, t)
            assert torch.allclose(x, x_rec, atol=1e-5), (
                f"Tall input test failed for nInput={nInput},
nOutput={nOutput}\n"
                f"x: {x}\nx_rec: {x_rec}"
            )
            print("Tall input test passed.")

        # Test 4: Batch behavior.
        nInput, nOutput = 6, 4
        layer = ReversibleDiagonalLayer(nInput, nOutput)
        x = torch.rand((7, nInput))
        if nInput > nOutput:
            x[:, nOutput:] = 0.0
        b, t = 1, 0
        y = layer.forward(x, b, t)
        x_rec = layer.reverse(y, b, t)
        assert torch.allclose(x, x_rec, atol=1e-5), (
            f"Batch behavior test failed for nInput={nInput},
nOutput={nOutput}\n"
            f"x: {x}\nx_rec: {x_rec}"
        )
        print("Batch behavior test passed.")

        print("All tests passed!")
class ReversibleLinearLayer(Layer):
    def __init__(self, nInput, nOutput, naive=False, hasBias=True,
stable=False):
        super(ReversibleLinearLayer, self).__init__(nInput, nOutput)
        self.naive = naive
        self.hasBias = hasBias
        self.rank = min(nInput, nOutput)
        self.stable = stable

        # U is a square rotation matrix on the input side (nInput ×
nInput)
        self.U = ReversibleRotationLayer(dim=nInput, naive=self.naive)
        # V is a square rotation matrix on the output side (nOutput ×
nOutput)
        self.V = ReversibleRotationLayer(dim=nOutput,
naive=self.naive)
        # Sigma now builds an (nInput × nOutput) diagonal matrix.
        self.Sigma = ReversibleDiagonalLayer(nInput, nOutput)

        if self.naive:
            self.noise = nn.Parameter(torch.randn(nInput, nOutput))
        if self.hasBias:
            self.bias = nn.Parameter(torch.zeros(1, nOutput))
            self.biasNoise = nn.Parameter(torch.randn(1, nOutput))

```

```

def compute_W(self):
    # Compute full weight:  $W = U \cdot \Sigma \cdot V^T$ .
    U_matrix = self.U.rotation_matrix() # shape: (nInput, nInput)
    # Build  $\Sigma$  of shape (nInput, nOutput)
    Sigma_matrix = torch.zeros(self.nInput, self.nOutput,
device=self.Sigma.lamda.device,
dtype=self.Sigma.lamda.dtype)
    for i in range(self.rank):
        Sigma_matrix[i, i] = self.Sigma.lamda[i]
    V_matrix = self.V.rotation_matrix() # shape: (nOutput,
nOutput)
    # W: (nInput, nOutput)
    W = U_matrix @ Sigma_matrix @ V_matrix.T
    return W
def compute_Winverse(self):
    # Compute pseudoinverse:  $W_{inv} = V \cdot \Sigma^{-1} \cdot U^T$ .
    U_matrix = self.U.rotation_matrix() # shape: (nInput, nInput)
    Sigma_inv = torch.zeros(self.nOutput, self.nInput,
device=self.Sigma.lamda.device,
dtype=self.Sigma.lamda.dtype)
    for i in range(self.rank):
        Sigma_inv[i, i] = 1.0 / self.Sigma.lamda[i]
    V_matrix = self.V.rotation_matrix() # shape: (nOutput,
nOutput)
    # W_inv: shape (nOutput, nInput)
    W_inv = V_matrix @ Sigma_inv @ U_matrix.T
    return W_inv
def forward(self, x, bias=1.0, temp=0.0):
    if self.stable:
        self.Sigma.stabilize()
    if self.naive:
        self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
        W = bias * self.compute_W() + temp * self.noise
        # Naive forward:  $y = x @ W$ 
        y = x @ W
    else:
        # Non-naive branch: apply U, then  $\Sigma$ , then  $V^T$ .
        xShape = list(x.shape)
        xShape[-1] = self.nOutput
        x = x.reshape(-1, self.nInput)
        x1 = self.U.forward(x, bias, temp) # shape: (batch,
nInput)
        x2 = self.Sigma.forward(x1, bias, temp) # shape: (batch,
nOutput); note:  $\Sigma$ .forward does  $x1 @ S$  where  $S \in \mathbb{R}^{(nInput \times nOutput)}$ 
        y = self.V.forward(x2, bias, temp)
        # Now apply  $V^T$ : since V is square, we multiply on the
right by  $V^T$ .
        # V_matrix = self.V.rotation_matrix() # shape: (nOutput,
nOutput)
        # y = x2 @ V_matrix.T # shape: (batch, nOutput)
        y = y.reshape(xShape)

```



```

        if self.hasBias:
            self.biasNoise = nn.Parameter(torch.randn(1,
self.nOutput))
            y += self.bias + temp * self.biasNoise
        return y
    def reverse(self, y, bias=1.0, temp=0.0):
        if self.hasBias:
            y -= self.bias + temp * self.biasNoise
        if self.naive:
            W_inv = self.compute_Winverse() + temp * self.noise.T
            # Naive reverse:  $x = y @ W\_inv$ 
            x = y @ W_inv
            self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
        else:
            yShape = list(y.shape)
            yShape[-1] = self.nInput
            y = y.reshape(-1, self.nOutput)
            # Non-naive reverse: undo  $V^T$ , then  $\Sigma$ , then  $U$ .
            x2 = self.V.reverse(y, bias, temp)
            # Undo  $V^T$ : multiply by  $V$ .
            #V_matrix = self.V.rotation_matrix() # shape: (nOutput,
nOutput)
            #x2 = y @ V_matrix # shape: (batch, nOutput)
            # Undo  $\Sigma$ : recover x1 using  $\Sigma$ .reverse
            x1 = self.Sigma.reverse(x2, bias, temp) # shape: (batch,
nInput)
            # Undo  $U$ :  $x = U$ .reverse(x1)
            x = self.U.reverse(x1, bias, temp) # shape: (batch,
nInput)
            x = x.reshape(yShape)
        if self.hasBias:
            self.biasNoise = nn.Parameter(torch.randn(1,
self.nOutput))
        return x

    @staticmethod
    def test():
        torch.manual_seed(42)
        nInput, nOutput = 7, 11
        layer = ReversibleLinearLayer(nInput=nInput, nOutput=nOutput,
naive=True)
        gLayer = ReversibleLinearLayer(nInput=nInput, nOutput=nOutput,
naive=False)

        # Disable noise completely for testing
        b = 1
        t = 0.000001
        # Create test input
        x = torch.rand((2, 5, nInput))
        print(f"Input shape: {x.shape}")

        # Test naive implementation

```

```

start_time = time.time()
y_naive = layer.forward(x, b, t)
x_naive_restored = layer.reverse(y_naive, b, t)
naive_error = torch.norm(x - x_naive_restored)
print(f"Naive output shape: {y_naive.shape}")
print(f"Naive reconstruction error: {naive_error}")
end_time = time.time()
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time} seconds")

# Test givens implementation
start_time = time.time()
y_givens = gLayer.forward(x, b, t)
x_givens_restored = gLayer.reverse(y_givens, b, t)
givens_error = torch.norm(x - x_givens_restored)
print(f"Givens output shape: {y_givens.shape}")
print(f"Givens reconstruction error: {givens_error}")
end_time = time.time()
elapsed_time = end_time - start_time
print(f"Elapsed time: {elapsed_time} seconds")

# Compare outputs
output_diff = torch.norm(y_naive - y_givens)
print(f"Output difference: {output_diff}")

# Now test with assertions
assert naive_error < 0.001, f"Naive implementation failed with
error {naive_error}"
assert givens_error < 0.001, f"Givens implementation failed
with error {givens_error}"
print("All tests passed!")

class SigmaLayer(ErgodicLayer):
    def __init__(self, nInput, nOutput, permuteInput=False):
        super().__init__(nInput, nOutput, permuteInput=permuteInput)
        self.layer = LinearLayer(nInput, nOutput, hasBias=True)
        self.saturate = True
        self.activation = torch.zeros(1, nOutput, 1)

    def forward(self, x):
        bias, temp = self.bias, self.temperature
        x = self.permute(x)
        y = self.layer.forward(x, bias, temp) # (batch_size,
output_dim)
        if self.saturate:
            self.activation = torch.tanh(y)
            y = self.activation.clone()
        y = self.unpermute(y)
        return y

    @staticmethod
    def test():
        nInput, nOutput = 3, 4
        layer = SigmaLayer(nInput=nInput, nOutput=nOutput)

```

```

x = torch.randn((2, 5, nInput))
layer.setTemperature(0.001)

criterion = nn.MSELoss() # Mean Squared Error Loss
optimizer = optim.Adam(layer.getParameters(), lr=0.01) # Adam
Optimizer
optimizer.zero_grad() # Clear gradients
y = layer(x)
#y_inv = layer.reverse(y)

loss = criterion(y, y) # Compute loss
loss.backward() # Backpropagation

print(f"Original input: {x}")
print(f"After linear: {y}")
#print(f"Inverse operation result: {y_inv}")
class ReversibleSigmaLayer(SigmaLayer):
    def __init__(self, nInput, nOutput, naive=False,
permuteInput=False):
        super().__init__(nInput, nOutput, permuteInput=permuteInput)
        self.layer = ReversibleLinearLayer(nInput, nOutput,
naive=naive, hasBias=True)
    def reverse(self, y):
        y = self.permute(y)
        y = y.squeeze(0)
        if self.saturate:
            self.activation = torch.atanh(y) # this can be faster if
we keep the tanh activation
        y = self.activation.clone()
        x = self.layer.reverse(y, self.bias, self.temperature) #
(batch_size, output_dim)
        x = self.unpermute(x)
        return x

    @staticmethod
    def test():
        nInput, nOutput = 5, 7
        permute = False
        #naive = False
        layer = ReversibleSigmaLayer(nInput=nInput, nOutput=nOutput,
permuteInput=permute, naive=False)

        x = torch.randn((2, 5, nInput))
        layer.setTemperature(0.000000001)
        y = layer.forward(x)
        y_inv = layer.reverse(y)

        #print(f"Original input: {x}")
        #print(f"After reversible linear: {y}")
        #print(f"Inverse operation result: {y_inv}")
        assert(torch.norm(x-y_inv) < 0.00001)

```

```

        layer = ReversibleSigmaLayer(nInput=nInput, nOutput=nOutput,
permuteInput=False, naive=True)
        x = torch.randn((4, 8, nInput))
        layer.setTemperature(0.00000001)
        y = layer.forward(x)
        assert y.shape == (4,8,nOutput), "Incorrect Size"
        y_inv = layer.reverse(y)
        assert(torch.norm(x-y_inv) < 0.00001)
class PiLayer(ErgodicLayer):
    def __init__(self, nInput, nOutput, permuteInput=False):
        super().__init__(nInput, nOutput, permuteInput=permuteInput)
        self.weights = nn.Parameter(torch.zeros(nInput,
nOutput))
        self.noise = nn.Parameter(torch.randn(nInput,
nOutput))
        self.biasWeight = nn.Parameter(torch.zeros(1, 1,
self.nOutput)) # Per-output-feature bias
        self.biasWeightNoise = nn.Parameter(torch.randn(1, 1,
self.nOutput)) # Per-output-feature bias

        self.saturate = True
        self.hasBiasWeight = True
        self.useEpsilon = True

    def forward(self, x):
        bias, temp = self.bias, self.temperature
        x = self.permute(x)
        # This method implements PI(1+wx)
        # Tried to implement as exp( SIGMA(log(e) * log(wx)) ) so that
we could invert with standard methods,
        # but the matrix log(e) is not invertible. If we try simply
PI(w+x), we get a weight matrix with negative values,
        # so we can't take it into the log domain.
        if temp:
            self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
            self.biasWeightNoise = nn.Parameter(torch.randn(1,
self.nInput, self.nOutput)) # Per-output-feature bias
            w = bias * self.weights + temp * self.noise
            ndim = len(x.shape)
            assert self.nInput == x.shape[-1], "Incorrect shape in
piLayer"
            # x: shape (N, J, K), w: shape (K, L)
            # Expand x to (N, J, K, 1) and w to (1, 1, K, L) so they can
broadcast together:
            if ndim == 2:
                WX = x.unsqueeze(-1) * w.unsqueeze(0)
                if self.hasBias:
                    WX += (bias * self.biasWeight + temp *
self.biasWeightNoise)
                if self.saturate:
                    term = 1 + torch.tanh(WX) # shape (N, J, K)
                else:
                    term = 1 + WX

```

```

        output = (self.certainty.squeeze(0)+ temp *
self.certaintyNoise.squeeze(0)) * torch.prod(term, dim=1) # result
has shape (N, K, L)
        output = self.unpermute(output)
    else:
        # x: shape (N, J, K), w: shape (K, L)
        # Expand x to (N, J, K, 1) and w to (1, 1, K, L) so they
can broadcast together:
        x2 = x.unsqueeze(-1)
        w2 = w.unsqueeze(0)
        WX = x2 * w2
        if self.hasBiasWeight:
            WX += (bias*self.biasWeight.unsqueeze(1) +
temp*self.biasWeightNoise.unsqueeze(1))
        if self.saturate:
            term = 1 + torch.tanh(WX) # shape (N, J, K)
            if self.useEpsilon:
                term += epsilon
        else:
            term = 1 + WX
        # Compute the product along the J dimension:
        output = torch.prod(term, dim=2) # result has shape
(N, K, L)
        output = self.unpermute(output)
    return output

    @staticmethod
    def test():
        nBatch, nInput, nOutput = 5, 3, 4
        layer = PiLayer(nInput=nInput, nOutput=nOutput,
permuteInput=True)

        # x must be positive
        x = torch.randn((nBatch, nInput, 6))
        layer.setTemperature(0.001)
        y = layer(x)
        assert(y.shape==(nBatch, nOutput, 6))

        print(f"Original input: {x}")
        print(f"After linear: {y}")
    @staticmethod
    def xorTest():
        X = torch.tensor([
            [0, 0], [0, 1], [1, 0], [1, 1]], dtype=torch.float32)
        Y = torch.tensor([
            [0], [1], [1], [0]], dtype=torch.float32)
        X = X.unsqueeze(2)
        Y = Y.unsqueeze(2)
        nInput, nHidden, nOutput = (2,3,1)
        pi = PiLayer(nInput, nHidden, permuteInput=True) #
Hidden layer using PiLayer
        sigma = SigmaLayer(nHidden, nOutput, permuteInput=True) #
Output layer using SigmaLayer

```

```

        criterion = nn.MSELoss() # Mean Squared Error Loss
        optimizer = optim.Adam(chain(pi.parameters(),
sigma.parameters()), lr=0.01) # Adam Optimizer
        epochs = 1000
        sigma.setTemperature(0.0001)
        pi.setTemperature(0.0001)
        for epoch in range(epochs):
            optimizer.zero_grad() # Clear gradients

            x1 = pi(X) # Pass through PiLayer
            y = sigma(x1) # Pass through SigmaLayer

            loss = criterion(y, Y) # Compute loss
            loss.backward() # Backpropagation
            optimizer.step() # Update weights
            if epoch % 100 == 0:
                print(f'Epoch {epoch}/{epochs}, MSE:
{loss.item():.6f}')
class ReversiblePiLayer(ErgodicLayer):
#     • This computes:
#  $y_j = b_j \prod_i \left(1 - \tanh(w_{ji} x_i)\right)$ 
#  $z_j = b_j \prod_i \left(1 + \tanh(w_{ji} x_i)\right)$ 
#     • These match up with:
#  $\gamma_j = \frac{1}{2} \log\left(\frac{z_j}{y_j}\right) = \sum_i \tanh^{-1}(\tanh(w_{ji} x_i)) = \sum_i w_{ji} x_i = (W x)_j$ 
#     • And thus:
#  $x \approx W^\dagger \gamma$ 
# The full inversion process is:
# 1. Define:
#  $s_j := \sum_i \tanh^{-1}(w_{ji} x_i) = W x$ 
# 2. In forward pass, use:
#  $y_j = b_j \prod_i (1 - \tanh(w_{ji} x_i))$ ,  $\text{quad}$ 
#  $z_j = b_j \prod_i (1 + \tanh(w_{ji} x_i))$ 
# 3. In reverse pass, compute:
#  $\gamma_j = \frac{1}{2} \log\left(\frac{z_j}{y_j}\right) = \sum_i \tanh^{-1}(\tanh(w_{ji} x_i)) = \sum_i w_{ji} x_i$ 
#  $\Rightarrow \gamma = W x$ 
# 4. Finally:
#  $x = W^\dagger \gamma$ 

    def __init__(self, nInput, nOutput, naive=False,
permuteInput=False, hasBias = True):
        super().__init__(nInput, nOutput, permuteInput=permuteInput)
        self.naive = naive
        self.hasBias = hasBias
        self.useEpsilon = True
        if not self.naive:
            assert 2*nInput == nOutput, "There must be twice as many
outputs as inputs for invertibility when using the non-naive
algorithm."
            self.saturate = True # saturation is currently necessary for
a smooth inversion
            # W: shape (out_features, in_features)
            if naive:

```

```

        self.W      = nn.Parameter(torch.randn(nInput, nOutput))
        self.noise   = nn.Parameter(torch.randn(nInput, nOutput))
    else:
        self.layer   = ReversibleLinearLayer(nInput, nOutput,
naive=True)
        self.biasWeight = nn.Parameter(torch.zeros(1, 1,
self.nOutput)) # Per-output-feature bias
        self.biasNoise  = nn.Parameter(torch.randn(1, 1,
self.nOutput)) # Per-output-feature bias

    def forward(self, x):
        bias, temp = self.bias, self.temperature

        """
        x: (batch_size, in_features)
        Output: yz: (batch_size, out_features, 2)
        y_j = b_j * prod_i (1 - tanh(w_{ji} * x_i))
        z_j = b_j * prod_i (1 + tanh(w_{ji} * x_i))
        """
        if temp != 0:
            if self.naive:
                self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
                self.biasNoise = nn.Parameter(torch.randn(1, 1,
self.nOutput)) # Per-output-feature bias
                x = self.permute(x)

            if not self.naive:
                W = self.layer.compute_W().T # XXX seems not to use bias,
variance
                W = (bias * W + temp * self.noise)
                WX = x.unsqueeze(-1) * W.unsqueeze(0).unsqueeze(0)
            else:
                W = (bias * self.W + temp *
self.noise).unsqueeze(0).unsqueeze(0)
                WX = x.unsqueeze(-1) * W # (batch,
out_features, in_features)
                if self.hasBias:
                    WX = WX + (bias*self.biasWeight.unsqueeze(1) +
temp*self.biasNoise.unsqueeze(1))

            # Compute tanh(w_{ji} * x_i)
            sWX = torch.tanh(WX)
            # Apply (1 - tanh(...)) and (1 + tanh(...))
            one_minus = 1 - sWX
            one_plus  = 1 + sWX
            if self.useEpsilon:
                one_minus += epsilon
                one_plus  += epsilon
            # Product over input dimension (i)
            y = torch.prod(one_minus, dim=2)
            z = torch.prod(one_plus, dim=2)
            #s = torch.concatenate((y, z), dim=1) # (batch, out_features,
2)

```

```

        stacked = torch.stack((y,z), dim=1)
        interleaved = torch.flatten(stacked, start_dim=1, end_dim=2)
        y = self.unpermute(interleaved)
        return y

    def reverse(self, yz):
        bias, temp = self.bias, self.temperature
        """
        Reverse pass:  $x \approx W^\dagger * \gamma$ , where  $\gamma_j = 0.5 * \log((1 + \tanh(w \cdot x)) / (1 - \tanh(w \cdot x))) = Wx$ 
        """
        yz = self.permute(yz)
        n2 = round(yz.shape[1]/2)
        uninterleaved = torch.unflatten(yz, 1, (2,n2))
        y = uninterleaved[:,0,:].squeeze(1)
        z = uninterleaved[:,1,:].squeeze(1)
        gamma = 0.5 * torch.log(z / y)
        if self.hasBias:
            gamma = gamma - torch.sum(bias*self.biasWeight +
temp*self.biasNoise, dim=1).unsqueeze(1) # (batch, out_features)
        if not self.naive:
            W_pinv = self.layer.compute_Winverse().T
            x = gamma @ W_pinv
        else:
            W_pinv = torch.linalg.pinv( (self.W + temp*self.noise) )
# (in_features, out_features)
            x = gamma @ W_pinv
            x = self.unpermute(x)
            if temp != 0:
                if self.naive:
                    self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))
                    self.biasNoise = nn.Parameter(torch.randn(1, 1,
self.nOutput)) # Per-output-feature bias
            return x

    @staticmethod
    def test():
        nBatch      = 16
        nInput       = 3
        nOutput      = 2 * nInput
        nFeatures    = 5

        layer = ReversiblePiLayer(nInput=nInput, nOutput=nOutput,
naive=True, hasBias=True, permuteInput=True)
        x = torch.randn(nBatch, nInput, nFeatures)
        layer.setTemperature(0.00000001)
        yz = layer.forward(x)
        print("Forward output shape:", yz.shape) # Should be (batch,
out_features, 2)
        x_recon = layer.reverse(yz)
        print("Reconstructed x shape:", x_recon.shape) # Should be
(batch, in_features)

```



```

        error = torch.norm(x - x_recon) / torch.norm(x)
        print(f"Reconstruction relative error: {error.item():.6f}")
        assert error < 0.1, f"Reconstruction error too high: {error}"
        print("ReversiblePiLayer test passed.")

class VQLayer(Layer):
    nOutput = 0

    def __init__(self, dim, codebookSize, numQuantizers):
        super(VQLayer, self).__init__(dim, dim)
        self.vq = ResidualVQ(
            dim=dim,
            codebook_size=codebookSize,
            num_quantizers=numQuantizers,
            decay=0.8, # the exponential moving average decay, lower
means the dictionary will change faster
            commitment_weight=1.0, # the weight on the commitment
loss
            use_cosine_sim=True,
            rotation_trick=True # Set False to use the STE gradient
estimator or True to use the rotation trick.
        )

    def distance(self, x, y):
        # This is a Euclidean distance
        return torch.sqrt(torch.sum((x - y) ** 2))

    def forward(self, x, t=0):
        batch = len(x)
        # percepts are Batch x nOutput x nFeatures
        x = x.reshape((-1, self.nInput))
        quantized, indices, commit_loss, all_codes = self.vq(x,
return_all_codes=True)
        distances = all_codes.permute(1, 2, 0)
        # need to produce
        # Find the closest prototype for each input
        # closest_prototype_indices = torch.argmin(distances, dim=1)
        # Get the labels of the closest prototypes
        # predicted_labels = self.labels[closest_prototype_indices]
        return all_codes

    def reverse(self, y, t=0):
        raise ValueError("Value not computed")
        return x

class DecisionBoundaryLayer(Layer):
    def __init__(self, nInput, nOutput, learning_rate=0.01):
        super(DecisionBoundaryLayer, self).__init__(nInput, nOutput)
        self.learning_rate = learning_rate
        self.weight = nn.Parameter(torch.zeros(nInput,
nOutput))
        self.noise = nn.Parameter(torch.randn(nInput,
nOutput))

```

```

def forward(self, x, t=0):
    if t != 0:
        self.noise = nn.Parameter(torch.randn(self.nInput,
self.nOutput))

    W = self.weight + t*self.noise
    dot_product = torch.matmul(x, W)
    decision = torch.where(dot_product >= 0, torch.tensor(1.0,
device=x.device),
                           torch.tensor(-1.0, device=x.device))
    return decision

def update(self, x, t=0):
    d1 = torch.norm(x - self.weight) ** 2
    d2 = torch.norm(x + self.weight) ** 2
    if d1 < d2:
        self.weight.data += self.learning_rate * (x.unsqueeze(1) -
self.weight.data)
    else:
        self.weight.data -= self.learning_rate * (x.unsqueeze(1) +
self.weight.data)

    @staticmethod
    def test():
        n_points = 100
        data = torch.randn(n_points, 2)
        data[:, 0] *= 1.5

        layer = DecisionBoundaryLayer(nInput=2, nOutput=1,
learning_rate=0.01)
        for _ in range(1000):
            idx = torch.randint(0, n_points, (1,))
            x = data[idx].squeeze()
            layer.update(x)

        w = layer.weight.detach().cpu().numpy()
        w_neg = -w

        data_np = data.numpy()
        plt.figure(figsize=(8, 6))
        plt.scatter(data_np[:, 0], data_np[:, 1], label="Data",
alpha=0.6)

        plt.quiver(0, 0, w[0], w[1], angles='xy', scale_units='xy',
scale=1, color='r', linestyle='-', label='w')
        plt.quiver(0, 0, w_neg[0], w_neg[1], angles='xy',
scale_units='xy', scale=1, color='b', linestyle='-', label='-w')

        a, b = w
        x_vals = np.linspace(np.min(data_np[:, 0]) - 1,
np.max(data_np[:, 0]) + 1, 100)
        if np.abs(b) > 1e-5:
            y_vals = - (a / b) * x_vals

```

```

        plt.plot(x_vals, y_vals, color='g', linestyle='dashed',
label='Hyperplane')
    else:
        plt.axvline(0, color='g', linestyle='dashed',
label='Hyperplane')

    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Decision Boundary Learning')
    plt.legend()
    plt.grid(True)
    plt.xlim(np.min(data_np[:, 0]) - 2, np.max(data_np[:, 0]) + 2)
    plt.ylim(np.min(data_np[:, 1]) - 2, np.max(data_np[:, 1]) + 2)
    warnings.filterwarnings('ignore', message='.*line style')
    plt.show()
class NormLayer(Layer):
    pNorm      = 2
    removeBias = True
    takeExp     = False
    dim         = 1
    lr          = 0.001

    def __init__(self, nInput, nOutput, pNorm=2, exp=False):
        """
        Hand-coded implementation of Layer Normalization or Softmax-
        like operation.
        """
        super().__init__(nInput, nOutput)
        self.pNorm = pNorm
        self.exp    = exp
        self.dim    = 1
        # predict the bias and variance within this layer
        self.W      = nn.Parameter(torch.zeros(2))
        #self.noise  = nn.Parameter(torch.zeros(2)) If is ErgodicLayer

    def forward(self, x):
        #bias, temp = self.bias, self.temperature
        shape = x.shape
        N = shape[self.dim]
        mean = x.mean(dim=self.dim, keepdim=True)
        if self.pNorm == 2:
            assert(self.exp == False)
            norm = torch.sum((x - mean) ** 2, dim=self.dim,
keepdim=True) / (N - 1)
            norm = torch.sqrt(norm + epsilon)
            moments = torch.concat([mean, norm], axis=self.dim)
            #error    = self.W - moments
            #self.W    = nn.Parameter((1-self.lr) * self.W + (self.lr)
* error)
            moments -= self.W
            x_norm  = (x - moments[:, 0:1]) / moments[:, 1:2]
        else:
            assert(self.pNorm == 1)
            if self.exp:

```

```

        exp_x = torch.exp(x)
        sum_exp = exp_x.sum(dim=self.dim, keepdim=True)
        x_norm = exp_x / (sum_exp + epsilon)
        moments = sum_exp
    else:
        sum_x = torch.sum(x, dim=self.dim, keepdim=True)
        x_norm = x / (sum_x + epsilon)
        moments = sum_x
    assert(False, "False until we add the W prediction in this
code branch")
    # append mean and norm
    output = torch.concat((x_norm, moments), dim=-1)
    return output

def reverse(self, y):
    """
    Reverse operation (only defined for pNorm=2 without softmax).
    """
    if self.pNorm != 2:
        raise NotImplementedError("Reverse only supported for
pNorm=2.")
    x = y[:, 0:-2]
    mean = y[:, -2:-1] + self.W[0:1]
    norm = y[:, -1:] + self.W[1:2]
    return x * norm + mean

@staticmethod
def test():
    torch.manual_seed(42)

    print("== Testing NormLayer ==")
    # === Test 1: pNorm=2 (Standard LayerNorm) ===
    x = torch.randn(10, 20)
    manual_layer_norm = NormLayer(20, 22, pNorm=2)
    manual_layer_norm.lr = 0
    normalized = manual_layer_norm(x)

    # Shape check
    #assert normalized.shape == x.shape, "Output shape mismatch"
    # Mean check
    # assert
    torch.allclose(normalized.mean(dim=manual_layer_norm.dim),
torch.zeros_like(normalized.mean(dim=manual_layer_norm.dim)),
atol=1e-5), "Mean not close to 0"
    # Variance check
    # assert
    torch.allclose(normalized.var(dim=manual_layer_norm.dim,
unbiased=True),
torch.ones_like(normalized.var(dim=manual_layer_norm.dim)),
atol=1e-5), "Variance not close to 1"

    # === Reverse test ===
    reconstructed = manual_layer_norm.reverse(normalized)

```

```

        assert torch.allclose(x, reconstructed, atol=1e-5), "Reverse
reconstruction failed"

    print("✓ pNorm=2 forward + reverse passed.")

    # === Test 2: pNorm=1, exp=True (Softmax mode) ===
    x = torch.randn(10, 5)
    builtin_softmax = nn.Softmax(dim=1)
    manual_softmax = NormLayer(5, 7, pNorm=1, exp=True)

    builtin_output = builtin_softmax(x)
    manual_output = manual_softmax(x)[:,-1]

    # Shape check
    assert builtin_output.shape == manual_output.shape, "Softmax
output shape mismatch"

    # Softmax values check
    assert torch.allclose(builtin_output, manual_output,
atol=1e-6), "Softmax outputs differ"

    # Sum-to-1 check
    assert
torch.allclose(manual_output.sum(dim=manual_softmax.dim),
torch.ones(x.shape[0]), atol=1e-6), "Softmax output does not sum to 1"

    # Range check
    assert torch.all((manual_output >= 0) & (manual_output <= 1)),
"Softmax output contains values outside [0, 1]"

    print("✓ Softmax mode passed.")

    # === Test 3: 3D input ===
    #x_3d = torch.randn(5, 10, 20)
    #manual_layer_norm_3d = NormLayer(2, pNorm=2)
    #normalized_3d = manual_layer_norm_3d(x_3d)
    # 3D shape check
    #assert normalized_3d.shape == x_3d.shape, "3D input
normalization failed"
    #print("✓ 3D input passed.")
    #print("All NormLayer tests passed successfully!")
class AttentionLayer(Layer):
    def __init__(self, nInput, nOutput, nHidden=None,
symmetric=False):
        super(AttentionLayer, self).__init__(nInput, nOutput)
        if not nHidden:
            self.nHidden = nOutput
        else:
            self.nHidden = nHidden
        self.mask = None
        self.beta = 10
        # self.dropout = nn.Dropout(p=0.1)
        self.symmetric = symmetric

```

```

self.reversible = False
#self.target_norm = target_norm
#self.tol = tol
#self.maxIter = maxIter

if self.symmetric:
    self.A = LinearLayer(self.nInput, self.nHidden)
    self.V = LinearLayer(self.nInput, self.nHidden)
else:
    self.Q = LinearLayer(self.nInput, self.nHidden)
    self.K = LinearLayer(self.nInput, self.nHidden)
    self.V = LinearLayer(self.nInput, self.nHidden)
    self.Out = LinearLayer(self.nHidden, self.nOutput)
def create_mask(self, sentences):
    self.mask = torch.zeros(self.nOutput, self.objectSize,
dtype=torch.bool)
    for i, s in enumerate(sentences):
        self.mask[i, len(s):] = True

def forward(self, x, bias=1, temp=0):
    if self.symmetric:
        a2 = self.A(x, bias, temp)
        value = x if self.nHidden == self.nInput else self.V(x,
bias, temp)
        scores = torch.matmul(a2.transpose(-2, -1), a2) /
(self.nInput ** 0.5)
    else:
        query = self.Q(x, bias, temp)
        key = self.K(x, bias, temp)
        value = x if self.nHidden == self.nInput else self.V(x,
bias, temp)
        scores = torch.matmul(query.transpose(-2, -1), key) /
(self.nInput ** 0.5)

    if self.mask is not None:
        scores = scores.masked_fill(self.mask == 0, float('-inf'))

    # Hopfield networks repeat until convergence here
    #for _ in range(self.maxIter):
    #    norm = torch.norm(attention, p=2, dim=-1).mean()
    #    if torch.abs(norm - self.target_norm) < self.tol:
    #        break
    #    attention = F.softmax(self.beta * scores, dim=-1)

    if not self.reversible:
        attn = F.softmax(self.beta * scores, dim=-1)
    else:
        attn = scores

    # attn = self.dropout(attn)
    output = value @ attn
    if self.nHidden != self.nOutput:
        output = self.Out(output, bias, temp)
    return output

```

```

@staticmethod
def test():
    """
    Static test for sanity check.
    """
    #torch.manual_seed(42)
    nInput = 6
    nOutput = 3
    layer = AttentionLayer(nInput=nInput, nOutput=nOutput,
nHidden=7)

    x = torch.randn(4, 5, nInput) # batch of 4
    y = layer.forward(x, bias=1, temp=0)

    #print("Input minus output:")
    #print(x-x_rec)
    #assert torch.norm(x-x_rec) < 1, "Norm too high"
#endregion

#region Activation Functions
class Activation:
    """
    ACTIVATION A set of functions that apply fuzzy logic operations to
    NN activations.
    """

    # Constants
    strictParts = True
    activationThreshold = 0.01

    @staticmethod
    def positive(x):
        return (x > 0) * x

    @staticmethod
    def negative(x):
        return -(x < 0) * x

    @staticmethod
    def neutral(x):
        return 1 - np.abs(x)

    @staticmethod
    def true():
        return 1

    @staticmethod
    def false():
        return -1

    @staticmethod
    def unknown():
        return 0

```

```

# Determine if the concept is active
@staticmethod
def isActive(x):
    return np.abs(x) >= Activation.activationThreshold

# Determine equality
@staticmethod
def isEqual(x1, x2):
    return np.all(x1 == x2)

@staticmethod
def isPart(x1, x2):
    tf = Activation.sign(x1) * Activation.sign(x2) *
(np.minimum(np.abs(x1), np.abs(x2)) / np.abs(x1))
    # Remove NaN values (from dividing by zero)
    tf = tf[~np.isnan(tf)] # equivalent to tf(isnan(tf)) = []

    if len(tf) == 0:
        tf = 1 # nothing is a part of everything
    else:
        tf = np.mean(tf)
        if Activation.strictParts and np.abs(tf) < 1:
            tf = 0
    return tf

@staticmethod
def isWhole(x1, x2):
    tf = Activation.sign(x1) * Activation.sign(x2) * np.abs(x1) /
np.maximum(np.abs(x1), np.abs(x2))
    # Handle zero values
    tf[(x1 == 0) & (x2 != 0)] = 0 # zero values in x1 are not
wholes
    # Remove values where both x1 and x2 are zero
    tf = tf[~((x1 == 0) & (x2 == 0))] # equivalent to tf(x1==0 &
x2==0) = []

    if len(tf) == 0:
        tf = 1 # nothing is a part of everything
    else:
        tf = np.mean(tf)
        if Activation.strictParts and np.abs(tf) < 1:
            tf = 0
    return tf

@staticmethod
def isReducer(x1, x2):
    # x1 reduces x2 if abs(x2-x1) < abs(x2)
    return np.sum(np.abs(x2 - x1)) < np.sum(np.abs(x2))

@staticmethod
def part(x1, x2):
    return (Activation.sign(x1) + Activation.sign(x2)) / 2 *
np.minimum(np.abs(x1), np.abs(x2))

```



```

    @staticmethod
    def whole(x1, x2):
        y = (Activation.sign(x1) + Activation.sign(x2)) / 2 *
np.maximum(np.abs(x1), np.abs(x2))
        y[Activation.sign(x1) != Activation.sign(x2)] = np.nan
        return y

    @staticmethod
    def minMag(x1, x2):
        if np.abs(x1) <= np.abs(x2):
            return x1
        else:
            return x2

    @staticmethod
    def maxMag(x1, x2):
        if np.abs(x1) >= np.abs(x2):
            return x1
        else:
            return x2

    @staticmethod
    def error(x1, x2):
        return np.linalg.norm(x1 - x2)

    # signum, where sgn(0) = 1
    @staticmethod
    def sign(v1):
        y = np.sign(v1.astype(float) if hasattr(v1, 'astype') else
float(v1))
        y[y == 0] = 1
        return y

    @staticmethod
    def norm(x):
        return np.linalg.norm(x)

    # saturate to [-1..1]
    @staticmethod
    def saturate(x):
        if np.isnan(x):
            return 0
        else:
            return min(1, max(-1, x))

    # threshold to one of [-1,0,1]
    @staticmethod
    def threshold(x):
        if np.abs(x) < Activation.activationThreshold:
            return 0
        else:
            return x

```

```

@staticmethod
def complement(x):
    return Activation.sign(x) - x

@staticmethod
def negation(x):
    return -x

# symbolic activation should be in the range -1..1
@staticmethod
def convertSensation(x):
    return 2 * x - 1

@staticmethod
def test():
    def neg(x):
        # y = 1-x;
        return -x

    def scale(x):
        # y = (x+1)/2;
        return x

    def combine(x1, x2):
        sgn = (np.sign(x1) + np.sign(x2)) / 2
        amp = np.multiply(np.abs(x1), np.abs(x2))
        # y = sgn.*amp;
        # y = x1.*x2;
        y = np.mean([x1, x2], axis=0)
        y = np.sign(x1) * np.sign(x2) * (np.minimum(np.abs(x1),
np.abs(x2)) / np.abs(x1))
        return y

    x = np.arange(0, 2 * np.pi, 0.01)
    y1 = scale(np.cos(x - np.pi / 6))
    y2 = scale(np.sin(x))

    plt.clf()
    plt.bar(x, y1, color='red', alpha=0.5)
    #plt.hold(True) # Note: plt.hold is deprecated in newer
matplotlib versions
    plt.bar(x, y2, color='blue', alpha=0.5)
    plt.bar(x, combine(y1, y2), color='black', alpha=0.5)
    plt.show()
#endregion

#region Error Functions
class CertaintyWeightedMAELoss(nn.Module):
    def __init__(self, alpha=0.5):
        super().__init__()
        self.alpha = alpha

    def forward(self, predictions, targets):
        abs_error = torch.abs(targets - predictions)

```

```

        certainty = torch.abs(predictions)
        loss = abs_error * (self.alpha * certainty + (1 - self.alpha))
        return torch.mean(loss)
class CertaintyWeightedMSELoss(nn.Module):
    def __init__(self, alpha=0.5):
        super().__init__()
        self.alpha = alpha

    def forward(self, outputs, targets):
        """
        outputs: [batch_size, num_outputs]
        targets: [batch_size, num_outputs] (for regression or one-hot
for classification)
        """
        # Certainty: magnitude of the prediction for each sample
        certainty = outputs.abs().sum(dim=1) # [batch_size], or you
could use .mean(dim=1)
        mse_loss = ((outputs - targets) ** 2).sum(dim=1) #
[batch_size]
        cw_mse_loss = mse_loss * certainty #
[batch_size]
        hybrid_loss = self.alpha * cw_mse_loss.mean() + (1 -
self.alpha) * mse_loss.mean()
        #hybrid_loss = self.alpha * cw_mse_loss + (1 - self.alpha) *
mse_loss
        return hybrid_loss
class CertaintyWeightedCrossEntropy(nn.Module):
    def __init__(self, alpha=0.5, epsilon=1e-8):
        super().__init__()
        self.alpha = alpha
        self.epsilon = epsilon

    def forward(self, logits, targets):
        # If targets are one-hot, convert to indices
        if targets.dim() == 2 and targets.size(1) == logits.size(1):
            targets = targets.argmax(dim=1)
        # Ensure targets are int64 and on the same device as logits
        targets = targets.to(dtype=torch.long, device=logits.device)
        batch_size = logits.size(0)
        batch_indices = torch.arange(batch_size, device=logits.device)

        log_probs = F.log_softmax(logits, dim=1) # [batch,
num_classes]
        ce_loss = -log_probs[batch_indices, targets]
        #norm_logits = torch.norm(logits, dim=1)
        #cwce_loss = -norm_logits * log_probs[batch_indices, targets]
        p_true = torch.exp(log_probs[batch_indices, targets])
        cwce_loss = -p_true * log_probs[batch_indices, targets] #
possibly erroneous
        hybrid_loss = self.alpha * cwce_loss + (1 - self.alpha) *
ce_loss
        #norm = torch.norm(hybrid_loss, p=2)
        return hybrid_loss.mean()
#endregion

```

```

#region Various kinds of memory
# Base class for various kinds of (temporally-varying) Memory
class Mem:
    def __init__(self, sz=None):
        self.lr = 0.01
        self.nTrials = 0
        self.output = None
        self.reset(sz)

    def delta(self, *args):
        """Increment nTrials by one. Extra arguments are ignored."""
        self.nTrials += 1
    def get(self):
        """Return the current output array."""
        return self.output
    def set(self, in_val):
        """Set the output array to a new value."""
        self.output = in_val
    def reset(self, sz=None):
        """
        Reset the output to a zeros array and nTrials to 0.
        If sz is not provided, use the current output's shape (or (0,
0) if undefined).
        """
        if sz is None:
            if self.output is None:
                sz = (0, 0)
            else:
                sz = self.output.shape
        self.output = torch.zeros(sz)
        self.nTrials = 0
    def removeRC(self, r=None, c=None):
        """
        Remove a row and/or column from the output.
        The row (r) and column (c) indices are assumed to be given in
MATLAB 1-indexed form.
        """
        if r is not None:
            self.output = np.delete(self.output, r - 1, axis=0)
        if c is not None:
            self.output = np.delete(self.output, c - 1, axis=1)
    def insertRC(self, r=None, c=None):
        """
        Insert a row and/or column of zeros into the output.
        The indices r and c are assumed to be 1-indexed.
        """
        if r is not None:
            new_row = np.zeros((1, self.output.shape[1]))
            self.output = np.insert(self.output, r - 1, new_row,
axis=0)
        if c is not None:
            new_col = np.zeros((self.output.shape[0], 1))

```

```

        self.output = np.insert(self.output, c - 1, new_col,
axis=1)
    def setRC(self, r, c, val):
        """
        Set the (r, c) element to val and its symmetric element (c, r)
to val.
        Indices are assumed to be 1-indexed.
        """
        self.output[r - 1, c - 1] = val
        self.output[c - 1, r - 1] = val

    @staticmethod
    def test():
        """
        Test method that creates instances of several Mem-derived
classes
        and calls their testImpulse method.
        """
        names = ['ExponentialMem', 'GammaMem', 'MeanMem']
        for name in names:
            # Create an instance using globals() (similar to MATLAB's
feval)
            m = globals()[name]()
            Mem.testImpulse(m, name)

    @staticmethod
    def testOne(m):
        """
        Test one impulse by incrementing and plotting the output.
        """
        plt.figure(1)
        plt.clf()
        x = np.arange(1, 1001)
        y = np.zeros(1000)
        m.delta(1, 1) # extra parameters are ignored
        for i in range(len(x)):
            y[i] = m.get()
            m.delta(1, 1)
        plt.plot(x, y)
        plt.title("Test One")
        plt.show()

    @staticmethod
    def testImpulse(m, name):
        """
        Test impulse response by incrementing and plotting the output.
        """
        plt.figure(1)
        x = np.arange(1, 1001)
        y = np.zeros(1000)
        m.delta(1.0)
        for i in range(len(x)):
            y[i] = m.get()
            m.delta(0)

```

```

        plt.plot(x, y)
        plt.title(f"Impulse Response: {name}")
        plt.show()
# ZOHHMem subclass: Zero-Order Hold memory.
class ZOHHMem(Mem):
    def __init__(self, sz=1):
        super().__init__(sz)

    def delta(self, in1, in2=None):
        # Call the base class delta and then set output to in1.
        super().delta()
        self.output = in1
# StateMem subclass: adds a 'state' property.
class StateMem(Mem):
    def __init__(self, sz=1):
        super().__init__(sz)
        self.state = None
        self.reset(sz)

    def reset(self, sz=None):
        """
        Reset output and state to zeros and nTrials to 0.
        """
        if sz is None:
            if self.output is None:
                sz = (0, 0)
            else:
                sz = self.output.shape
        super().reset(sz)
        self.state = torch.zeros(sz)

    def delta(self, *args):
        # Just call the base class delta.
        super().delta(*args)

    def removeRC(self, r=None, c=None):
        """
        Remove a row/column from both output and state.
        """
        super().removeRC(r, c)
        if r is not None:
            self.state = np.delete(self.state, r - 1, axis=0)
        if c is not None:
            self.state = np.delete(self.state, c - 1, axis=1)

    def insertRC(self, r=None, c=None):
        """
        Insert a row/column of zeros into both output and state.
        """
        super().insertRC(r, c)
        if r is not None:
            new_row = np.zeros((1, self.state.shape[1]))
            self.state = np.insert(self.state, r - 1, new_row, axis=0)
        if c is not None:

```

```

        new_col = np.zeros((self.state.shape[0], 1))
        self.state = np.insert(self.state, c - 1, new_col, axis=1)

    def setRC(self, r, c, val):
        """
        Set the (r,c) element in both output and state (and
        symmetrically).
        """
        super().setRC(r, c, val)
        self.state[r - 1, c - 1] = val
        self.state[c - 1, r - 1] = val
# RLSTMem subclass: Recursive Least Squares memory.
class RLSTMem(StateMem):
    def __init__(self, sz=1):
        super().__init__(sz)
        self.momLR = 0.2

    def delta(self, in1, in2=None):
        # Call the base class (Mem) delta.
        Mem.delta(self)
        # Compute error using the L2 norm.
        err = np.linalg.norm(self.output - in1, 2)
        self.output = self.output + err * in1
        # Optionally update state (commented out in original code):
        # self.state = self.state + self.momLR * err * in1
        # self.output = self.output + self.state
# ProbMem subclass: probabilistic memory update.
class ProbMem(Mem):
    def __init__(self, sz=1):
        super().__init__(sz)

    def delta(self, in1, in2):
        super().delta()
        # Iterate over the indices of in1 and in2.
        for r in range(len(in1)):
            for c in range(len(in2)):
                # Increase or decrease conditional probability based
                on Activation.
                if Activation.true(in2[c]) and
                Activation.true(in1[r]):
                    self.output[r, c] = ((self.nTrials - 1) /
                    self.nTrials) * self.output[r, c] + (1 / self.nTrials) * 1
                elif Activation.true(in2[c]) and
                Activation.false(in1[r]):
                    self.output[r, c] = ((self.nTrials - 1) /
                    self.nTrials) * self.output[r, c] + (1 / self.nTrials) * -1
# MeanMem subclass: computes a running mean.
class MeanMem(Mem):
    def __init__(self, sz=1):
        super().__init__(sz)

    def delta(self, in1, in2=None):
        super().delta()

```

```

        self.output = ((self.nTrials - 1) / self.nTrials) *
self.output + (1 / self.nTrials) * in1
# GammaMem subclass: blends state with output using a second learning
rate.
class GammaMem(StateMem):
    def __init__(self, sz=1, lr2=0.05):
        super().__init__(sz)
        self.lr2 = lr2

    def delta(self, in1, in2=None):
        # Call the StateMem delta method.
        StateMem.delta(self)
        self.state = (1 - self.lr) * self.state + self.lr * in1
        self.output = (1 - self.lr2) * self.output + self.lr2 *
self.state

    @staticmethod
    def test():
        Mem.testOne(GammaMem())
# ExponentialMem subclass: exponential memory update.
class ExponentialMem(Mem):
    def __init__(self, sz=1, lr=None):
        super().__init__(sz)
        if lr is not None:
            self.lr = lr

    def delta(self, in1, in2=None):
        super().delta()
        self.output = (1 - self.lr) * self.output + self.lr * in1
    @staticmethod
    def test():
        # Create an instance of ExponentialMem and run a test.
        m_exp = ExponentialMem(sz=(5, 5), lr=0.1)
        # Simulate a delta update with an input (for example, a 5x5
array).
        x = np.ones((5, 5))
        m_exp.delta(x)
        x = x = np.zeros((5, 5))
        m_exp.delta(x)
        print("ExponentialMem output after two delta calls:")
        print(m_exp.get())
# CorrMem subclass: correlational memory update.
class CorrMem(Mem):
    def __init__(self, sz=1):
        super().__init__(sz)

    def delta(self, in1, in2):
        super().delta()
        for r in range(len(in1)):
            for c in range(len(in2)):
                val = in1[r] * in2[c]
                # Avoid division by zero.
                denom = np.sqrt(in1[r]**2 * in2[c]**2)
                if denom != 0:

```



```

        val = Activation.saturate(val / denom)
    else:
        val = 0
    amt = max(abs(in1[r]), abs(in2[c]))
    self.output[r, c] = ((self.nTrials - amt) /
self.nTrials) * self.output[r, c] + (amt / self.nTrials) * val
#endregion

# Example usage:
if __name__ == "__main__":
    torch.autograd.set_detect_anomaly(True)

    LinearLayer.test()
    ReversibleRotationLayer.test()
    ReversibleDiagonalLayer.test()
    ReversibleLinearLayer.test()

    ReversiblePiLayer.test()
    SigmaLayer.test()
    ReversibleSigmaLayer.test()
    PiLayer.test()
    PiLayer.xorTest()
    ReversiblePiLayer.test()

    AttentionLayer.test()
    NormLayer.test()
    Mem.test()
    DecisionBoundaryLayer.test()

```